



Linguaggi

Università di Verona
Imbriani Paolo -VR500437
Prof.ssa Isabella Mastroeni

May 19, 2026

Contents

1	Introduzione	5
2	Linguaggi di programmazione	5
2.1	Linguaggi formali	6
2.2	Aspetti di progettazione	6
2.3	Tipi di linguaggi di programmazione	7
2.3.1	Paradigmi di programmazione	7
3	Implementazione di un linguaggio di programmazione	8
3.1	Macchina astratta per il linguaggio di programmazione	8
3.2	Interprete	9
3.2.1	Struttura dell'interprete	9
3.3	Compilatore	11
3.3.1	Struttura del compilatore	12
3.4	Specializzazione	13
4	Descrivere i linguaggi di programmazione	13
4.1	La sintassi	14
4.1.1	Strumenti di def. della sintassi	15
4.1.2	Definizione non formale	16
4.2	L'analisi semantica per le espressioni	17
4.2.1	Induzione strutturale	18
4.3	Semantica	23
4.3.1	Composizionalità della semantica	26
4.3.2	Equivalenza di programmi	26
4.4	Categorie sintattiche	27
4.4.1	Linguaggio "giocattolo" PLO	30
5	IMP	32
5.1	Espressioni	32
5.1.1	Aspetti implementativi	34
5.2	Identificatori	36
5.2.1	Aspetti implementativi	36
5.2.2	Bindings	36
5.2.3	Tipi di bindings	37
5.2.4	Ambienti in IMP	38
5.2.5	Scope	39
5.3	Tipi	40

5.3.1	Ambiente statico	40
5.3.2	Semantica statica	41
5.3.3	Ambienti compatibili	41
5.4	Dichiarazioni	42
5.4.1	Sintassi	42
5.4.2	Aggiornamento di un ambiente	42
5.4.3	Composizione sequenziale	43
5.4.4	Composizione privata	43
5.4.5	Identificatori in posizione libera e di definizione	44
5.4.6	Analisi semantica per le dichiarazioni	45
5.4.7	Semantica dinamica	46
5.5	Variabili	50
5.5.1	Locazione	50
5.5.2	Semantica statica per le dichiarazioni con variabili	52
5.5.3	Aggiornamento della memoria	53
5.5.4	Semantica dinamica per le espressioni con identificatori variabili	53
5.5.5	Semantica dinamica per le dichiarazioni con variabili	55
5.6	Comandi	56
5.6.1	Sintassi dei comandi	57
5.6.2	Semantica statica dei comandi	57
5.6.3	Semantica dinamica dei comandi	58
5.6.4	Semantica dell'if	60
5.6.5	Composizione	62
5.6.6	Comando iterativo	63
5.7	Blocco	65
5.7.1	Blocco inline	65
5.7.2	Regola di visibilità	66
5.7.3	Tempo di vita	67
5.7.4	Semantica statica	69
5.7.5	Semantica dinamica	69
5.8	Procedure	70
5.8.1	Ambiente di riferimento di una procedura	72
5.8.2	Regole di scoping	73
5.8.3	Scoping statico	74
5.8.4	Scoping dinamico	79
5.9	Allocazione della memoria	83
5.9.1	Allocazione dinamica su pila	84
5.9.2	Procedura di chiamata a funzione	84
5.9.3	Implementazione dello scoping statico	85

5.10	Resoluzione dei riferimenti non locali	86
5.11	Implementazione scoping dinamico	87
5.12	Procedure in IMP	88
5.12.1	Regole per le dichiarazioni	89
5.12.2	Regole per la chiamata	89
5.13	Passaggio dei parametri	90
5.13.1	Modalità di passaggio di parametri: direzioni	91
5.13.2	Passaggio per valore	92
5.13.3	Passaggio per riferimento	93
5.14	Funzioni di ordine superiore	95
5.14.1	Regole di Binding	95
5.15	Ricorsione	97
5.15.1	Ricorsione in coda	99

1 Introduzione

Un linguaggio interpretato è un linguaggio di programmazione in cui le istruzioni vengono eseguite direttamente, senza la necessità di una fase di compilazione. Un interprete è rappresentato da un modello di esecuzione cioè un CFG che rappresenta il flusso di controllo del programma. L'obiettivo è di studiare i linguaggi di programmazione e capire cosa li accomuna (insieme alle loro macro-differenze) e quale è il loro ruolo all'interno dello sviluppo. L'idea ora è capire quali cambiamenti sono stati necessari introdurre per arrivare a linguaggi di programmazione più evoluti, e capire quali sono i problemi che si sono incontrati nel corso del tempo e come sono stati risolti.

2 Linguaggi di programmazione

Da una parte abbiamo:

- **Dati:** che può essere qualsiasi cosa, numeri, stringhe qualcosa da manipolare
- **Algoritmi:** ovvero il calcolo da eseguire sui dati

📌 Definizione 2.1: Linguaggio di programmazione

Un **linguaggio di programmazione** è uno strumento formale per descrivere algoritmi applicati ai dati.

📌 Definizione 2.2: Programma

Un **programma** è una rappresentazione sintattica (che rispetta regole sintattiche del linguaggio di programmazione) finita di comportamenti potenzialmente infiniti. Questi comportamenti vengono definiti attraverso delle funzioni:

$$f : Dom \rightarrow Cod \cup \{\uparrow\}$$

(la funzione calcolata dall'algoritmo rappresentata/implementata attraverso il programma).

Descriviamo prima però cosa è un algoritmo:

📌 Definizione 2.3: Algoritmo

Sequenza di passi primitivi finita descritta attraverso un programma in un linguaggio di programmazione.

2.1 Linguaggi formali

Esistono diversi tipi di linguaggi formali:

- **Linguaggio matematico**: notazione rigorosa per rappresentare funzioni:

$$f(x) = x + 2$$

Da questo linguaggio è nato un paradigma quello **funzionale** (λ -calcolo, ML, Haskell, Lisp...)

- **Linguaggio logico**: notazione rigorosa, regole e assiomi, calcolo come dimostrazione. Questo linguaggio però manca la descrizione di calcoli potenzialmente infiniti. Anche da qui nasce un paradigma, quello **logico** (Prolog).
- **Linguaggio di programmazione**: notazione rigorosa (grammatica), regole (semantica delle istruzioni), istruzioni che permettono di descrivere i passi di calcolo.

Dalle esigenze di diversi tipi di applicazioni, sono nati diversi tipi di linguaggi di programmazione:

- Applicazioni scientifiche come **FORTRAN**
- Applicazioni economiche come **COBOL**
- Applicazioni AI come **LISP**
- Applicazioni di sistema come **C**
- Applicazioni web come **Javascript, Java, ...**

2.2 Aspetti di progettazione

Uno degli aspetti più importanti nella progettazione di un linguaggio di programmazione sono:

- **Readability**:
 - Sintassi chiara
 - No ambiguità
 - Facilità di lettura che rende facile comprendere i programmi
 - Facilità di analisi del codice
- **Writeability**:
 - Facilità di scrittura, quanto è facile/agevole
 - Scrivere programmi in quel linguaggio, facilità di analisi del codice
- **Reliability**:

- Conformità alle specifiche
- Riduzione di inserimento di errori di programmazione
- **Cost:**
 - Costo di efficienza
 - Costo di apprendimento
 - Costi di utilizzo in modo corretto
 - ...

2.3 Tipi di linguaggi di programmazione

Ci sono diversi tipi di linguaggi di programmazione:

- **Linguaggi di basso livello:** vicini al linguaggio macchina come:
 - Linguaggio macchina
 - Linguaggio assembly
- **Linguaggi di alto livello:** più vicini al linguaggio umano come:
 - Linguaggi funzionali
 - Linguaggi logici
 - Linguaggi imperativi (l'unione con i linguaggi funzionale si chiama "funzionali impuri") dove vengono definite variabili come astrazione di una cella di memoria

In questi linguaggi le variabili:

- Nei linguaggi **imperativi** è un'astrazione di una cella di memoria, che può essere modificata
- Nei linguaggi **funzionali** e **matematica** è una variabile matematica e quindi **immutabile**

2.3.1 Paradigmi di programmazione

Nei linguaggi ci sono i seguenti paradigmi:

- **Object-oriented:** dove i programmi sono organizzati in classi che rappresentano oggetti, con attributi e metodi.
- **Procedurali:** dove i programmi sono organizzati in procedure o funzioni che eseguono compiti specifici.
- **Strutturata:** dove i programmi sono organizzati in blocchi di codice
- **Dichiarativi:** si basano sulle basi matematiche e logiche, dove i programmi sono organizzati in dichiarazioni che descrivono ciò che deve essere fatto, piuttosto che come farlo.

3 Implementazione di un linguaggio di programmazione

Per implementare un linguaggio di programmazione, è necessario:

- **Architettura** della macchina su cui è implementato
- **Metodologie di progettazione**: cambiare da un linguaggio ad oggetti a un linguaggio dichiarativo determina cambiamenti nella progettazione del linguaggio e quindi anche nella sua implementazione

3.1 Macchina astratta per il linguaggio di programmazione

Abbiamo bisogno di una macchina astratta per il linguaggio di programmazione, che è un modello di esecuzione che rappresenta il flusso di controllo del programma.

Definizione 3.1: Macchina astratta

Dato un LP L , una M_L (macchina astratta per L) è un insieme di **strutture dati** e **algoritmi** che permettono di memorizzare e eseguire programmi scritti in L . Intuitivamente, M_L è la macchina il cui linguaggio è esattamente L .

Definizione 3.2: Linguaggio macchina

Dato una macchina M il linguaggio L_m (linguaggio macchina di M) è il linguaggio che ha come stringhe quelle direttamente interpretabili da M .

Si ha una pila di macchine astratte M_{i-1}, M_i, M_{i+1} e i linguaggi macchina $L_{m_{i-1}}, L_{m_i}, L_{m_{i+1}}$. Il linguaggio L_{m_i} può essere usato ai livelli superiori.

Teorema 3.1.1

Per ogni M_L esiste un unico linguaggio che è il linguaggio riconosciuto ed eseguito da M_L (linguaggio macchina di M_L). Per ogni LP possono esistere infinite macchine astratte per L .

Se immaginiamo le macchine astratte a più livelli, le macchine astratte di livello più basso donano i loro servizi ai livelli superiori che usano i linguaggi ad alto livello per implementare funzioni più complesse. Questo compongono diversi **livelli di astrazione**.

$$\text{Implementare } L \equiv \text{Realizzare } M_L$$

Per realizzare M_L si può:

- Realizzare M_L vuol dire tradurre L in un linguaggio macchina sottostante. (Compilazione)
- Oppure si può eseguire L sulla macchina sottostante. (Interpretazione)

📌 Definizione 3.3: Compilazione

Dati due linguaggi L, L' una **compilazione** vuol dire fare una traduzione esplicita tra L e L' o viceversa.

📌 Definizione 3.4: Interpretazione

Dati due linguaggi L, L' una **interpretazione** vuol dire fare una traduzione implicita tra L in L' ovvero simuliamo l'esecuzione di L attraverso L' .

3.2 Interprete

Per definire cosa è un interprete dobbiamo introdurre le seguenti notazioni:

- $Prog^L$: insieme dei programmi scritti in L
- D : insieme dei dati manipolati dai programmi in L
- P^L un singolo programma in L . (Quindi $P^L \in Prog^L$)

📌 Definizione 3.5

Dato L un linguaggio di programmazione, un **interprete** per L (scritto in L_0) è un programma

$$I^{LoL} : (Prog^L \times D) \rightarrow D$$

L'interprete è la macchina di Turing universale.

$$\underbrace{I^{LoL}(P^L, input) = output \text{ t.c. } P^L(input) = output}_{I^{LoL}(P^L, input) = P^L(input)}$$

3.2.1 Struttura dell'interprete

Vediamo ora come un generico interprete è strutturato, e quali sono le sue componenti principali. Lo scriveremo in pseudocodice, e vedremo che è composto da tre fasi principali: Fetch, Decode, Execute.

```
go := true;
while go do
    fetch (opcode, opinfo) from PC;
```

```

decode (opcode, opinfo);
if opccode needs arguments
    then fetch arguments from PC;
case opcode of
    opcode1: execute opcode1;
    opcode2: execute opcode2;
    ...
    opcodeN: execute opcodeN;
    halt: go := false;

if opcode has res then store(res)
pc = pc + sizeof(opcode) + sizeof(arguments);

```

Esempio 3.1

Un **autointerprete** è un interprete scritto nello stesso linguaggio che interpreta. Vediamo un esempio di codice:

```

1 input P, d; // Programma che deve essere interpretato e il suo dato
2 pc := 2; store := [in -> d, out -> 0, x_1 -> 0,...]
3 while pc < length(P) do {
4     instruction := lookup(P, pc); // Trova l'istruzione da eseguire
5     case instruction of // Dispatch on syntax
6         skip : pc++;
7         x := e : store := store [x -> eval(e, store)]; pc++;
8         ... endw;
9 }
10 output(e, store) = case e of
11     constant: e
12     variable: store(e)
13     e1 + e2: eval(e1, store) + eval(e2, store)
14     e1 - e2: eval(e1, store) - eval(e2, store)
15     e1 * e2: eval(e1, store) * eval(e2, store)
16     ...
17 end

```

3.3 Compilatore

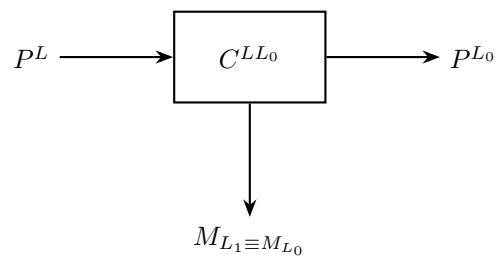
🔗 Definizione 3.6: Compilatore

Un compilatore da L a L_0 è un programma

$$C^{LL_0} : Prog^L \rightarrow Prog^{L_0}$$

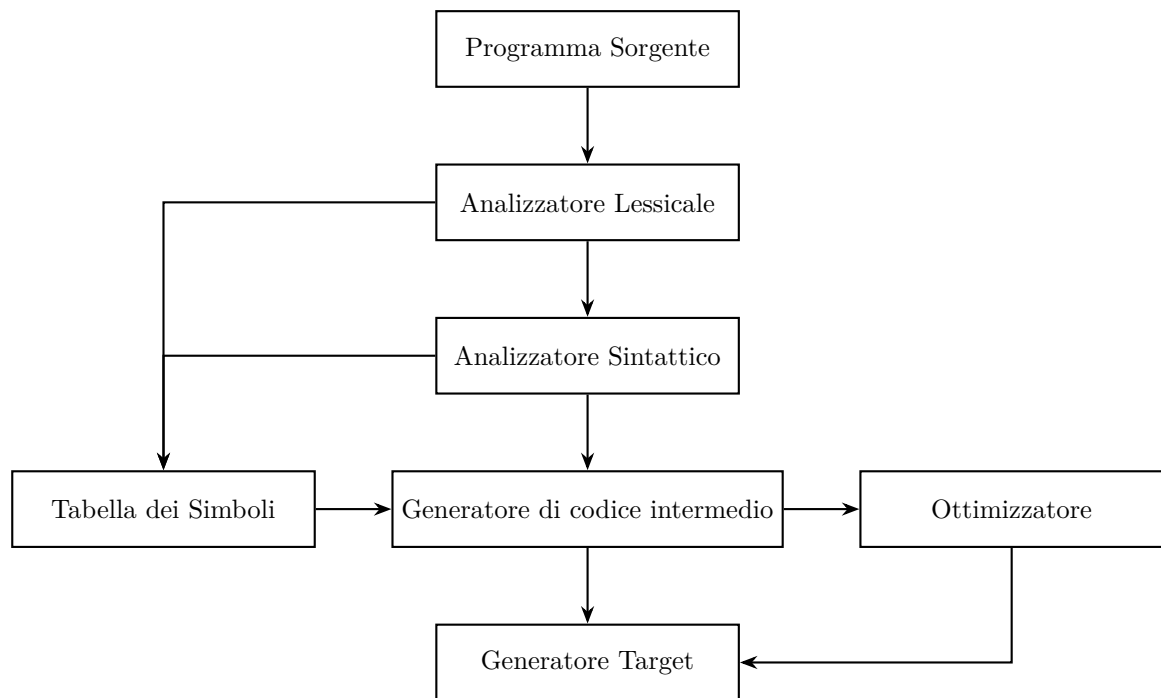
Dove

$$\forall P^L \in Prog^L . C^{LL_0}(P^L) = P^{L_0} \text{ t.c. } \forall d \in D . P^L(d) = P^{L_0}(d)$$



Traduce in un codice diverso e separa la traduzione dall'esecuzione.

3.3.1 Struttura del compilatore



Dove rispettivamente:

- Il programma sorgente è il programma scritto in L che deve essere compilato
- L'analizzatore lessicale è la fase che prende in input il programma sorgente e lo suddivide in token, ovvero unità lessicali come parole chiave, identificatori, operatori, ecc.
- L'analizzatore sintattico prende i token prodotti dall'analizzatore lessicale e costruisce una rappresentazione ad albero del programma, chiamata albero sintattico o AST (Abstract Syntax Tree).
- Il generatore di codice prende l'albero sintattico e produce un codice intermedio o direttamente il codice target.
- La tabella dei simboli è una struttura dati che memorizza informazioni sui simboli utilizzati nel programma, come variabili, funzioni, classi, ecc. Viene utilizzata durante l'analisi semantica e la generazione del codice.
- l'ottimizzatore migliora il codice generato in modo da renderlo più efficiente, ad esempio eliminando codice morto, riducendo il numero di istruzioni, migliorando l'uso della memoria, ecc.

- il generatore target prende il codice intermedio e lo traduce in codice eseguibile per la macchina target,

Ci sono alcune modifiche che agevolano l'ultima parte di generazione del codice e che ottimizza il codice generato in modo da renderlo più efficiente.

Esempio 3.2

Considerando M_L con L PL da implementare e M_0 con L_0 PL eseguibile.

L'interprete:

$$M_L \Leftarrow M_i \Leftarrow M_0$$

Il compilatore:

$$M_L \Rightarrow M_i \Rightarrow M_0$$

3.4 Specializzazione

Definizione 3.7: Specializzatore

Uno specializzatore per un PL L è un programma che realizza la funzione:

$$Spec_L : Prog^L \times D \rightarrow Prog^L$$

tale che

$$\forall P^L \in Prog^L, \forall d \in D$$

$$Spec_L(P^L, d) = P_d^L \text{ t.c. } P^L(d, y) = P_d^L(y)$$

Osservazione 3.1

Lo specializzatore fornisce un legame tra interprete e compilatore. Specializzando un interprete (scritto in L_0) sul programma (Scritto in L) otteniamo un **compilatore** da L a L_0 . Dove I è scritto in L_0 :

$Spec(I, P^L)$ è un programma scritto in L_0 che integra nel codice l'esecuzione di PL

4 Descrivere i linguaggi di programmazione

I linguaggi di programmazione sono pur sempre qualcosa di simili a quelli naturali con la quale condividono alcuni elementi in comune come:

- **La sintassi:** l'insieme delle regole di formazione (determina quali sono le frasi ben formate nel linguaggio) descrive relazioni tra simboli;

- **La semantica:** Attribuzione del significato ai simboli (determina il significato delle frasi ben formate nel linguaggio) descrive relazioni tra simboli e significati;
- **La pragmatica:** regole di buon utilizzo del linguaggio;
- **L'implementazione:** esistono strumenti che eseguono frasi ben formate nel linguaggio, e che quindi ne permettono l'utilizzo (interpreti, compilatori).

4.1 La sintassi

📌 Definizione 4.1: Parola

La parola è definita come una stringa in un alfabeto finito Σ ed è un oggetto atomico.

📌 Definizione 4.2: Frase

La frase è definita come una sequenza di parole (ben formata ovvero che segue le regole della sintassi).

📌 Definizione 4.3: Linguaggio

Il **linguaggio** è un insieme di frasi ben formate.

✎ Esempio 4.1

Per esempio, in fondamenti avevamo visto i seguenti:

$$\text{Il linguaggio} = \{a^n b^n \mid n \in \mathbb{N}\} \text{ con } \Sigma = \{a, b\}$$

Dove le stringhe possono essere $\varepsilon, ab, a^2b^2, \dots$. Per i linguaggi di programmazione

$$\Sigma = \text{if, while, then, else}$$

Le stringhe sono i programmi (sequenza di parole) e il linguaggio è il nostro linguaggio di programmazione.

📌 Definizione 4.4: Lessema

Un **lessema** è una parola che ha significato (costituito da soli terminali nell'alfabeto)

Esempio 4.2

Nella frase "if exp then com else com":

- if, then, else, sono lessemi
- exp, com non sono lessemi (non hanno significato)

Definizione 4.5: Token

I **token** sono categorie sintattiche (simboli non terminali). Nell'esempio (4.2)

- if, then, else sono lessemi
- exp, com sono token (categorie sintattiche)

Esempio 4.3

Nella frase "if $x > 0$ then $x = x + 1$ else $x = x - 1$ ":

- if, then, else sono lessemi
- $x > 0$, $x = x + 1$, $x = x - 1$ sono token (categorie sintattiche)

4.1.1 Strumenti di def. della sintassi

Definizione 4.6: Riconoscitore

Il **riconoscitore** è uno strumento per il riconoscimento di stringhe appartenenti al linguaggio.

In Fondamenti, abbiamo visto come il riconoscitore più semplice per un linguaggio è rappresentato da un automa a stati finiti e li ritroviamo nell'analisi lessicale per riconoscimento dei lessemi.

Definizione 4.7: Generatore

Il **generatore** è uno strumento per la generazione di stringhe appartenenti al linguaggio.

Il generatore più semplice per un linguaggio è rappresentato da una grammatica, utilizzate nell'analisi sintattica dell'albero di parsing. Una grammatica CF per il PL è $G = (V, \Sigma, R, S)$ dove:

- V è un insieme finito di categorie sintattiche

- Σ è un insieme di vocabolario parole ($V \cap T = \emptyset$)
- R è un insieme di regole di produzione

$$A \rightarrow \alpha, A \in V, \alpha \in (V \cup T)^*$$

- $S \in V$ è la categoria della frase (programma)

Si utilizza la notazione BNF (CF) spesso usato per i PL:

- $\rightsquigarrow \equiv ::=$

Esempio 4.4

Supponiamo di avere una grammatica per le espressioni booleane:

$$G = \langle \{Exp\}, \{1, 0, not, and, or, (,)\}, P, Exp \rangle$$

Dove Exp può essere:

- $Exp \rightarrow 0, Exp \rightarrow 1,$
- $Exp \rightarrow not\ Exp, Exp\ and\ Exp, Exp\ or\ Exp$

quindi:

$$Exp \rightarrow 0 \mid 1 \mid (Exp\ or\ Exp) \mid (Exp\ and\ Exp) \mid (not\ Exp)$$

4.1.2 Definizione non formale

Un linguaggio si può definire anche in un modo non formale, definendo i costrutti base:

- Assegnamento
- Composizione sequenziale
- Comando iterativo (ciclo)
- Comando condizionale (if)

Il linguaggio è definito su livelli, partendo dal basso verso l'alto:

1. Notazione sulla grammatica
2. Linguaggio regolare per riconoscere gli identificatori e le parole riservate al linguaggio di programmazione.
3. Espressioni booleane che parte dai simboli definiti ai livelli sottostanti:

$$Exp \rightarrow 0 \mid 1 \mid (Exp\ or\ Exp) \mid (Exp\ and\ Exp) \mid (not\ Exp)$$

- Comandi atomici, ad esempio l'assegnamento. Utilizza identificatori, un simbolo d'assegnamento e un'espressione (booleana in questo caso):

$$A \rightarrow x := B$$

- Comandi, come per esempio l'if o un ciclo. Utilizzano i comandi atomici e le espressioni booleane:

$$C \rightarrow A \mid C; C \mid \text{while } B \text{ do } C \mid \text{if } B \text{ then } C \text{ else } C$$

- Programmi, cioè sequenze di comandi.

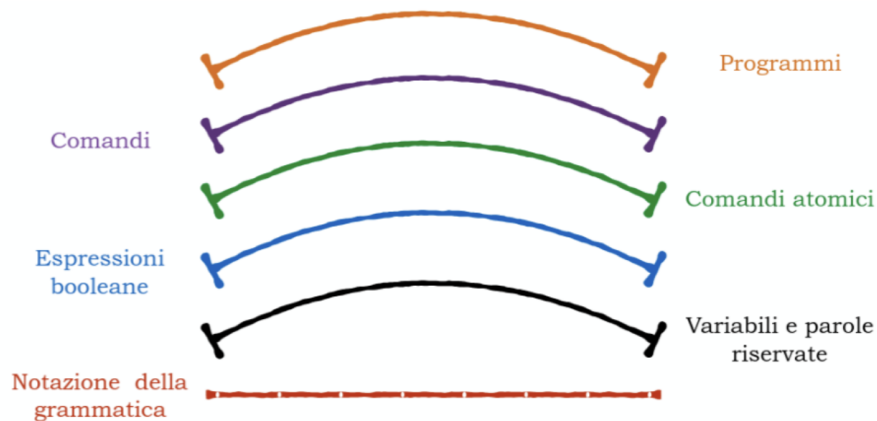


Figure 1: Rappresentazione grafica dei livelli di definizione del linguaggio

Si utilizzano espressioni parantesizzate per evitare ambiguità. Se non si utilizzassero le parentesi si potrebbe scegliere di eseguire prima un operatore invece di un altro, e questo porterebbe a risultati diversi dalla stessa espressione.

4.2 L'analisi semantica per le espressioni

L'analisi semantica verifica i vincoli sintattici, ad esempio che il numero di parametri formali sia uguale al numero di parametri attuali, oppure che l'uso di una variabile sia possibile solo dopo che sia stata dichiarata. Questi sono vincoli sul codice, quindi non vanno catturati dalla grammatica e si dicono **contestuali**.

- Sintassi $\rightarrow$ Grammatica CF
- Semantica $\rightarrow$ Tutti i processi di verifica ed esclusione che non sono descritti dalla sintassi.

4.2.1 Induzione strutturale

L'induzione strutturale permette di dimostrare proprietà di oggetti descritti in modo induttivo o ricorsivo. Una grammatica in modo induttivo il linguaggio (insieme di stringhe) che genera, quindi si può utilizzare l'induzione strutturale per dimostrare proprietà delle stringhe generate.

Definizione 4.8

Per dimostrare che una proprietà Π su tutti gli elementi di un insieme definito in modo strutturale da una grammatica bisogna:

1. Caso base: dimostrare la proprietà su tutti gli elementi di base, cioè quelli che sono esplicitamente inseriti nell'insieme di stringhe generate dalla grammatica.
2. Passo induttivo: dimostrare la proprietà sugli elementi composti sotto l'ipotesi induttiva che la proprietà valga per gli elementi che la compongono

Esempio 4.5

Definiamo l'insieme $X \subseteq \mathbb{N}$:

$$\begin{cases} \text{Base} & : 0 \in X \\ \text{Passo induttivo} & : \text{se } n \in X \text{ allora } n + 3 \in X \end{cases}$$

La proprietà è $X = 3\mathbb{N}$.

La proprietà è dimostrabile in modo strutturale è: $X \subseteq 3\mathbb{N}$:

1. Caso base: 0 è la base di X e $0 \in 3\mathbb{N}$
2. Passo induttivo: dimostriamo $\forall n. n \in X \rightarrow n \in 3\mathbb{N}$. Prendiamo $n \in X$, l'ipotesi induttiva è la seguente:

$$n \in X \rightarrow n \in 3\mathbb{N}$$

bisogna dimostrare che $n + 3 \in 3\mathbb{N}$. Ma sappiamo che:

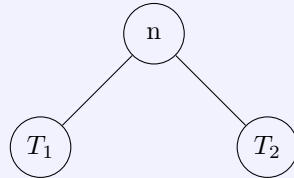
$$\begin{aligned} n \in 3\mathbb{N} &\Rightarrow \exists i. n = 3i \\ &\Rightarrow n + 3 = 3i + 3 = 3(i + 1) \in 3\mathbb{N} \end{aligned}$$

Esempio 4.6

Definiamo induttivamente gli alberi binari in cui i nodi sono numeri $n \in \mathbb{N}$. Ogni numero è un albero binario, perché è un albero con un solo nodo:

$$n \in BT \text{ (base)}$$

Consideriamo due alberi binari $T_1, T_2 \in BT$. allora:



Quindi:

$$br(T_1, T_2) \in BT$$

dove $br()$ è l'operatore che aggiunge un nodo ad una radice comune a T_1 e T_2 .

IMPORTANTE: Dimostriamo che la funzione che calcola il numero di foglie ($\#foglie(T)$) e la funzione che restituisce il numero di nodi che hanno figli ($\#branch(T)$) sono uguali a:

$$\forall T \in BT. \#foglie(T) = \#branch(T) + 1$$

Definiamo la funzione $\#foglie(T)$:

$$\begin{cases} \#foglie(n) &= 1 \\ \#foglie(br(T_1, T_2)) &= \#foglie(T_1) + \#foglie(T_2) \end{cases}$$

Definiamo la funzione $\#branch(T)$:

$$\begin{cases} \#branch(n) &= 0 \\ \#branch(br(T_1, T_2)) &= \#branch(T_1) + \#branch(T_2) + 1 \end{cases}$$

Dimostrazione per induzione sulla costruzione degli elementi di BT :

- Caso base: è il nodo che non ha figli in BT :

$$T = n \in BT$$

Sappiamo che

$$\#foglie(T) = \#foglie(n) = 1 = 1 + 0 = 1 + \#branch(T)$$

e questo soddisfa la proprietà.

- Passo induttivo: $T = br(T_1, T_2)$. Ipotesi induttiva: supponiamo che T_1 e T_2 , soddisfino la proprietà, quindi:

$$\#foglie(T_1) = \#branch(T_1) + 1 \quad \#foglie(T_2) = \#branch(T_2) + 1$$

Dobbiamo dimostrare che $\#foglie(T) = \#branch(T) + 1$, ovvero che:

$$\begin{aligned} \#foglie(T) &\stackrel{def T}{=} \#foglie(br(T_1, T_2)) \\ &\stackrel{def \#foglie}{=} \#foglie(T_1) + \#foglie(T_2) \\ &\stackrel{Hp ind}{=} \#branch(T_1) + 1 + \#branch(T_2) + 1 \\ &= \#branch(br(T_1, T_2)) + 1 \\ &= \#branch(T) + 1 \end{aligned}$$

Quindi anche T soddisfa la proprietà.

Esempio 4.7

Consideriamo la seguente grammatica:

$$S \rightarrow aa \mid bb \mid aSa \mid bSb$$

questa grammatica genera tutte le stringhe palindrome di lunghezza pari. Siccome aa e bb sono nel linguaggio, se si prende una qualsiasi stringa nel linguaggio S e si aggiungono a o b all'inizio e alla fine, si ottiene una stringa ancora palindroma, quindi anche questa stringa è nel linguaggio. Si può quindi definire questa grammatica in modo induttivo:

$$LS : \begin{cases} aa, bb, \in LS & \text{(Base)} \\ \sigma \in LS \Rightarrow a\sigma a, b\sigma b \in LS & \text{(Passo induttivo)} \end{cases}$$

Quindi $L(S) = LS$ e la proprietà da dimostrare è:

$$\sigma \in LS \Rightarrow \exists x, y. \sigma = xy \wedge x = reverse(y)$$

1. **Caso base:** Prendiamo:

$$aa = \sigma \Rightarrow x = a, y = a \quad \text{e} \quad x = a = \text{reverse}(a) = \text{reverse}(y)$$

analogo per $\sigma = bb$.

2. **Passo induttivo:** Prendiamo $\sigma \in LS$, l'ipotesi induttiva è la seguente:

$$\sigma = xy \text{ t.c. } x = \text{reverse}(y)$$

Prendiamo $\sigma' = a\sigma a$. Per ipotesi induttiva:

$$\sigma' = a\sigma a = a \overbrace{xy}^{\sigma} a = x'y'$$

con $x' = ax$ e $y' = ya$. Quindi:

$$\begin{aligned} x' &= ax = a\text{reverse}(y) \\ &= \text{reverse}(\underbrace{ya}_{y'}) \\ &= \text{reverse}(y') \end{aligned}$$

Abbiamo dimostrato che σ' è palindroma, quindi $\sigma' \in LS$.

Esempio 4.8

Consideriamo la seguente grammatica:

$$E \rightarrow n \mid E + E \mid E * E$$

1. Definire strutturalmente l'insieme Exp

2. Si considerino le seguenti funzioni:

- Conta il numero di operatori:

$$op(E) : op(n) = 0 \quad op(E_1 + E_2) = op(E_1 * E_2) = 1 + op(E_1) + op(E_2)$$

- Conta il numero di valori:

$$val(E) : val(n) = 1 \quad val(E_1 + E_2) = val(E_1 * E_2) = val(E_1) + val(E_2)$$

Dimostrare che:

$$\forall E . val(E) = op(E) + 1$$

Definizione strutturale dell'insieme Exp :

- **Base:** $n \in L(E)$
- **Passo induttivo:** se $e_1, e_2 \in Exp$ allora $e_1 + e_2, e_1 * e_2 \in Exp$. Le espressioni sono il minimo insieme chiuso che soddisfa queste condizioni, quindi non ci sono altre espressioni in $L(E)$. Quindi ad esempio:

$$3 + 5 * 2 \in Exp$$

Sappiamo che:

$$\begin{cases} val(n) = 1 \\ val(e_1 + e_2) = val(e_1 * e_2) = val(e_1) + val(e_2) \\ op(n) = 0 \\ op(e_1 + e_2) = op(e_1 * e_2) = op(e_1) + op(e_2) + 1 \end{cases}$$

Quindi sull'esempio precedente:

$$\begin{aligned} val(\underbrace{3+5}_{e_1} * \underbrace{2}_{e_2}) &= val(3+5) + val(2) \\ &= val(3) + val(5) + val(2) \\ &= 1 + 1 + 1 = 3 \end{aligned}$$

$$\begin{aligned} op(\underbrace{3+5}_{e_1} * \underbrace{2}_{e_2}) &= op(3+5) + op(2) + 1 \\ &= op(3) + op(5) + op(2) + 1 + 1 \\ &= 0 + 0 + 0 + 1 + 1 = 2 \end{aligned}$$

Dimostriamo che $\Pi : \forall e \in Exp . val(e) = op(e) + 1$ per induzione strutturale su Exp

- **Caso base:** Verifichiamo Π sugli elementi esplicitamente inseriti nell'insieme Exp , cioè sui numeri $n \in \mathbb{N}$: consideriamo le espressioni $e = n$

$$val(e) = val(n) = 1 = 1 + 0 = 1 + op(n) = 1 + op(e)$$

- **Ipotesi induttiva:** Dimostriamo per gli elementi composti supponendo l'ipotesi induttiva sugli elementi che li compongono. Sia $e = e_1 + e_2$ (ipotesi1) (analogo per $e = e_1 * e_2$), l'ipotesi induttiva è la seguente:

$$val(e_1) = op(e_1) + 1 \quad \wedge \quad val(e_2) = op(e_2) + 1$$

Dobbiamo dimostrare che $val(e) = op(e) + 1$:

$$\begin{aligned}
val(e) &\stackrel{ipotesi1}{=} val(e_1 + e_2) \\
&\stackrel{def\ val}{=} val(e_1) + val(e_2) \\
&\stackrel{Hp.\ Ind.}{=} \underbrace{op(e_1) + 1 + op(e_2) + 1}_{op(e_1 + e_2)} \\
&\stackrel{def\ op}{=} op(e_1 + e_2) + 1 \\
&\stackrel{ipotesi1}{=} op(e) + 1
\end{aligned}$$

Abbiamo dimostrato che II è vera per tutti gli elementi di Exp .

4.3 Semantica

La semantica è l'attribuzione di significato ai simboli. Il significato è un'entità autonoma che esiste indipendentemente dai simboli usati per descriverlo. Esistono 3 tipi di semantica:

- **Denotazionale:** È un modello completamente matematico che descrive il programma come una funzione su stati (programmi come funzioni). Lo **stato** descrive il valore di tutte le variabili in un certo momento.

$$Stato : Var \mapsto Val$$

La semantica denotazionale è una funzione che prende un programma e restituisce una funzione che prende uno stato iniziale e restituisce uno stato finale:

$$E : Prog \mapsto \underbrace{(Stato \rightarrow Stato)}_{\text{Denotazione associata al programma}}$$

Un esempio di semantica denotazionale è il comportamento input/output

- **Operazionale:** Descrive il significato del programma come sequenza di trasformazione di stati, **eseguendo** i suoi comandi sulla macchina astratta. Per semplificare vedremo lo stato come se fosse la memoria:

$$\text{Memoria: } \sigma : Var^n \mapsto Val^n$$

$$\text{Stato: } (Prog \times Mem) \cup Mem$$

dove:

- $Prog$ è il programma che stiamo eseguendo
- Mem è la memoria su cui stiamo eseguendo il programma

- $\cup Mem$ è l'insieme degli stati finali

Ad esempio:

Variabili: $\langle x, y, z \rangle \mapsto \langle 2, 5, 3 \rangle \rightsquigarrow \langle x \mapsto 2, y \mapsto 5, z \mapsto 3 \rangle$

Per eseguire un programma si utilizza un **sistema di transizione**:

$\langle \Sigma, \rightarrow \rangle$

dove:

- Σ è l'insieme degli stati
- $\rightarrow$ è la relazione di transizione che descrive come si passa da uno stato ad un altro eseguendo un comando del programma
- **Assiomatica**: Descrive il programma attraverso le proprietà che soddisfa. Si calcola utilizzando un sistema di derivazione che fornisce regole per dimostrare proprietà sui programmi:

Notazione: $\{P\}$ statement $\{Q\}$. Questa semantica si calcola mediante un sistema di prova: abbiamo una tripla da verificare e cerchiamo di dimostrarla applicando le regole.

$$\frac{\{P\} S \{Q\}, P' \Rightarrow P, Q \Rightarrow Q'}{\{P'\} S \{Q'\}}$$

$$\frac{\{P1\} S1 \{P2\}, \{P2\} S2 \{P3\}}{\{P1\} S1; S2 \{P3\}}$$

$$\frac{\{B \text{ and } P\} S1 \{Q\}, \{\text{not } B \text{ and } P\} S2 \{Q\}}{\{P\} \text{ if } B \text{ then } S1 \text{ else } S2 \{Q\}}$$

$$\frac{(I \text{ and } B) S \{I\}}{\{I\} \text{ while } B \text{ do } S \{I \text{ and } (\text{not } B)\}}$$

Figure 2: Esempio di sistema di derivazione per la semantica assiomatica

Quindi descrive relazioni tra proprietà di stati. Ad esempio fornendo le proprietà invarianti dei programmi, cioè le proprietà che sono vere prima e dopo l'esecuzione di un programma.

✎ Esempio 4.9

Consideriamo il seguente programma:

$$P = \begin{cases} z := 2; \\ y := z; \\ y := y + 1; \\ z := y; \end{cases}$$

Applichiamo i tre tipi diversi di semantica al programma P :

- **Semantica denotazionale:**

$$\begin{aligned} & \mathcal{D}[\overbrace{P}^{Prog}](\overbrace{z \mapsto \perp, y \mapsto \perp}^{Stato\ iniziale}) = \\ &= \mathcal{D}[z := 2; y := z; y := y + 1; z := y](z \mapsto \perp, y \mapsto \perp) \\ &= \mathcal{D}[z := y] \circ \mathcal{D}[y := y + 1] \circ \mathcal{D}[y := z] \circ \mathcal{D}[z := 2](z \mapsto \perp, y \mapsto \perp) \\ &= \mathcal{D}[z := y] \circ \mathcal{D}[y := y + 1] \circ \mathcal{D}[y := z](z \mapsto 2, y \mapsto \perp) \\ &= \mathcal{D}[z := y] \circ \mathcal{D}[y := y + 1](z \mapsto 2, y \mapsto 2) \\ &= \mathcal{D}[z := y](z \mapsto 2, y \mapsto 2) \\ &= (z \mapsto 3, y \mapsto 2) \end{aligned}$$

Quindi $\mathcal{D}[P](z \mapsto \perp, y \mapsto \perp) = (z \mapsto 3, y \mapsto 2)$ è usata per:

- Dimostrare proprietà (correttezza)
 - Fornisce uno strumento formale per ragionare sulle funzionalità dei programmi
 - Non è utile a chi usa il linguaggio di programmazione
- **Semantica assiomatica:** Fornisce proprietà relazionali sull'input e sull'output del programma, ad esempio:

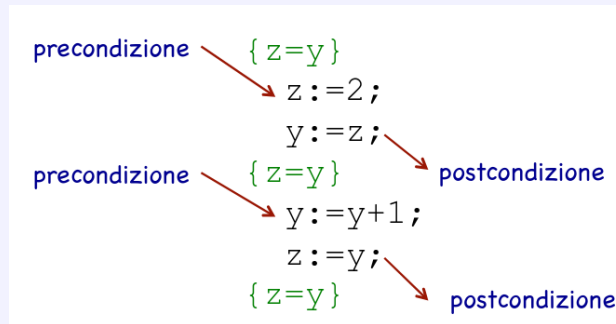


Figure 3: Esempio di proprietà assiomatica per il programma P

- **Semantica operativa:**

$$\begin{aligned}
& \langle P, (z \mapsto \perp, y \mapsto \perp) \rangle = \\
& = \langle z := 2; y := z; y := y + 1; z := y, (z \mapsto \perp, y \mapsto \perp) \rangle \\
& = \langle y := z; y := y + 1; z := y, (z \mapsto 2, y \mapsto \perp) \rangle \\
& = \langle y := y + 1; z := y, (z \mapsto 2, y \mapsto 2) \rangle \\
& = \langle z := y, (z \mapsto 2, y \mapsto 3) \rangle \\
& = \langle z \mapsto 3, y \mapsto 3 \rangle
\end{aligned}$$

4.3.1 Composizionalità della semantica

Il significato di ogni programma è funzione del significato dei suoi componenti immediati, cioè si possono ottenere le proprietà semantiche di un programma a partire dalle proprietà semantiche dei suoi componenti.

4.3.2 Equivalenza di programmi

La semantica viene utilizzata per verificare l'equivalenza tra programmi.

- $P_1 = P_2$ dice che P_1 è esattamente lo stesso programma di P_2 , cioè sono sintatticamente uguali. Questa si dice **uguaglianza sintattica**.
- $P_1 \equiv P_2$ dice che P_1 e P_2 hanno lo stesso comportamento (stesso significato), anche se sono scritti in modo diverso. Questa si dice **equivalenza** o uguaglianza semantica.

Esempio 4.10

Consideriamo i seguenti programmi:

- P_1 insertion sort
- P_2 bubble sort

Sappiamo che i due programmi sono diversi, quindi:

$$P_1 \neq P_2$$

Ma sappiamo che entrambi ordinano una lista, quindi il loro comportamento è lo stesso:

- Con la semantica denotazionale si ha:

$$\mathcal{D}[P_1] = \mathcal{D}[P_2] \rightsquigarrow P_1 \equiv P_2$$

- Con la semantica operativa si ha:

$$\llbracket P_1 \rrbracket \neq \llbracket P_2 \rrbracket \rightsquigarrow P_1 \neq P_2$$

Quindi l'equivalenza dipende dalla scelta della semantica

4.4 Categorie sintattiche

Le categorie sintattiche sono insiemi di stringhe che condividono una certa proprietà per:

- Manipolare dati (espressioni)
- Trasformare stati (comandi)
- Creare e manipolare i legami (dichiarazioni), ad esempio variabile-valore, variabile-tipo, funzione-codice, ecc...

Si hanno quindi:

- **Stato**: descrive gli effetti sugli identificatori
- **Ambiente**: è un insieme di legami tra identificatori e oggetti denotati

Le categorie sintattiche principali sono:

- **Espressioni**: denotano e manipolano valori. Le espressioni possono essere complesse e quindi devono essere **valutate** per restituire il singolo valore che sta denotando. Espressioni **sintatticamente diverse** possono denotare lo stesso valore. Ad esempio $1 + 5$ e $3 * 2$ denotano lo stesso valore.

Definizione 4.9

Due espressioni sono equivalenti se vengono valutate nello **stesso** valore a partire da qualunque stato/memoria iniziale.

Tra le espressioni ci sono quelle aritmetiche di cui gli aspetti progettuali sono:

- Arità delle operazioni (notazione)
- Regole di precedenza e associatività
- Ordine di valutazione (left-to-right, right-to-left, non specificato)
- Side effects (modificano la memoria)
- Overloading di operatori

- Espressioni con operandi di tipo misto. (i.e $1 + ciao$)
- Notazione:
 - * Infix: $1 + 5$
 - * Postfix: $1\ 5\ +$
 - * Prefix: $+ 1\ 5$

Quella prefissa e postfissa non richiedono regole di associatività e precedenza.

Esempio 4.11

Prendiamo la notazione **postfissa**. Immaginiamo di avere la seguente espressione:

$$5\ 3\ 2\ *\ +$$

- * Si legge 5, si inserisce nello stack (5)
- * Si legge 3, si inserisce nello stack (5, 3)
- * Si legge 2, si inserisce nello stack (5, 3, 2)
- * Si legge *, si esegue l'operazione con i due operandi in cima allo stack, quindi si esegue $3 * 2$ e si inserisce il risultato nello stack (5, 6)
- * Si legge +, si esegue l'operazione con i due operandi in cima allo stack, quindi si esegue $5 + 6$ e si inserisce il risultato nello stack (11)
- * Il risultato finale è 11

Esempio 4.12

Nel caso della notazione **prefissa**:

$$+\ * \ 3\ 2\ 5$$

Consideriamo c come l'arietà che memorizza il numero di operandi dell'operatore da applicare è 2, quindi:

- * Si legge +, si inserisce nello stack (+)
- * Si legge *, si inserisce nello stack (*, +) (l'arietà ora è $c = 2$)
- * Si legge 5, si inserisce nello stack (5, *, +) e si decrementa l'arietà di * di uno: $c = 1$.
- * Si legge 3, si inserisce nello stack (3, 5, *, +) e si decrementa l'arietà di * di uno: $c = 0$. Quindi si esegue $2 * 3$ e si inserisce il risultato nello stack

(6, +)

- * Si legge 5, si inserisce nello stack (5, 6, +) e si decrementa l'arietà di + di uno: $c = 0$.
- * Si esegue $5 + 6$ e si inserisce il risultato nello stack (11)

Esempio 4.13

Nel caso della notazione **infix**:

$$5 + 3 * 2$$

Le regole di **precedenza** servono per definire quale operatore applicare prima. Solitamente

- * $+, -$ sono allo stesso livello.
- * $*, /$ sono allo stesso livello e hanno precedenza su $+, -$.

Le regole di **associtività** decidono l'ordine di applicazione di operatori allo stesso livello di precedenza. Solitamente nella matematica è sempre left-to-right.

- **Comandi**: denotano richieste di modifica della memoria.
 - Le strutture di controllo decidono quali comandi eseguire
 - Gli assegnamenti modificano effettivamente la memoria

Rappresentano funzioni: $Memoria \mapsto Memoria$. I comandi vengono **eseguiti** per attuare la corrispondente richiesta di trasformazione della memoria. Queste trasformazioni sono **irreversibili**, cioè non possono essere cancellate, ma solo annullate semanticamente, ad esempio:

$$x := x + 1; x := x - 1$$

Questi comandi si annullano, ma lo stato è stato comunque modificato.

Definition 4.1. Due comandi sono equivalenti se per ogni stato iniziale in input producono lo stesso stato in output. Quindi l'equivalenza è vista come uguaglianza della funzione calcolata.

- **Dichiarazioni**: denotano richieste di creazione o modifica di legami, ovvero di **ambienti**. Le dichiarazioni vengono **elaborate** per attuare le richieste di creazione o modifica degli ambienti.

Definition 4.2. Due dichiarazioni sono equivalenti se producono lo stesso ambiente (e la stessa memoria) a partire da qualunque stato iniziale.

La separazione tra le categorie sintattiche non è rigida, ad esempio un'espressione può avere anche side effects, quindi modificare la memoria, ecc...

4.4.1 Linguaggio "giocattolo" PLO

In questo corso considereremo un linguaggio di programmazione "giocattolo" simile al linguaggio Pascal in cui:

- Esiste un singolo tipo di dato `integer`
- I booleani sono rappresentati come interi
- Strutture di controllo standard
- Astrazione del controllo
- Le procedure non hanno parametri

Il linguaggio è definito nel seguente modo:

```

<Program>  P → B
<Block>    B → DS
<Declar>   D → [const I = N{,I=N};]
            | [var I{,I};]
            | [procedure I;B;]
<Stmts>    S → ε | I := E
            | call I
            | if C then S
            | while C do S
            | begin S{;S} end
<Cond>     C → odd E | E > E
            | E >= E | E = E
            | E # E | E < E
            | E <= E
<Expr>     E → I | N | E op E | (E)

```

- I programmi *P* sono blocchi *B*.

- I blocchi B sono una dichiarazione D seguita da un comando S . Si noti che D ed S sono definiti in modo libero dal contesto, ovvero quello che possiamo generare da S non deve avere necessariamente nessun vincolo con quello che generiamo da D . In altre parole questa grammatica non ci obbliga in nessun modo ad usare identificatori precedentemente definiti. Quindi dalla descrizione sintattica rimangono fuori gli aspetti context sensitive (vincoli sintattici) che la grammatica CF non è in grado di rappresentare. Servirà la definizione di una “semantica statica” che avrà il ruolo di verificare questi vincoli prima di compilare e/o eseguire.
- Le dichiarazioni D sono dichiarazioni di valori costanti con nome (**const**), dichiarazioni di variabili (**var**) e dichiarazioni di procedure (ovvero di un blocco riferibile con un nome).
- I nomi sono gli identificatori I , considerati qui parte del vocabolario; N sono i numeri naturali, considerati anch’essi parte del vocabolario.
- Non abbiamo bisogno del concetto di tipo in quanto abbiamo solo un tipo di dato.
- Si noti che le produzioni di D sono tutte opzionali, questo significa che un programma può essere un blocco senza dichiarazioni.
- I comandi sono: il comando vuoto, l’assegnamento di un valore (denotato da una espressione) ad un identificatore, la chiamata ad una procedura mediante il suo nome, il comando condizionale che esegue un comando al verificarsi di una condizione booleana C , il ciclo che ripete un comando finché una data condizione C è vera e la composizione sequenziale di comandi.
- Le condizioni C sono espressioni dal valore booleano: **odd** è un predicato unario che verifica se l’espressione è 0, poi ci sono vari operatori di confronto di espressioni E .
- Le espressioni E sono identificatori, valori naturali, operazioni tra espressioni e espressioni tra parentesi.
- Si sintetizza con **op** la metavariable dell’insieme $\{+, -, *, /\}$. La stessa cosa si potrebbe fare per gli operatori di confronto, mettendo in C la produzione $E \text{ cop } E$ con *cop* metavariable per gli elementi in $\{>, >=, =, \#, <=, <\}$.

La grammatica delle espressioni è ambigua, ma più comoda da usare e si può utilizzare per definire la semantica. Per il compilatore però bisogna fornire una grammatica equivalente precisa e quindi

non ambigua, come la seguente:

```

<Expr>  E → [+,-] T [AT]
<Add op> A → + | -
<Terms> T → F [MF]
        M → * | /
<Factor> F → I | N | (E)
<Numbers> N → [+,-] d {d}
<Digit> d → 0 | ... | 9
<Ident> I → l {l, d}
<Letter> l → A | ... | Z

```

5 IMP

5.1 Espressioni

Definiamo la grammatica delle espressioni. Prendiamo dove $Eval = bool \cup int$ (quindi può essere sia un booleano che un intero):

$$\begin{aligned}
 E &\rightarrow E + E \mid E * E \mid (E) \mid n \\
 A &\rightarrow n \mid A \text{ op } A \quad \text{op} = \{+, -, *\} \\
 B &\rightarrow true \mid false \mid A = A \mid not B \mid B \text{ or } B
 \end{aligned}$$

Un po' di notazione:

- $\mathcal{N} \rightsquigarrow$ insieme dei valori numerici n, m, p
- $\mathcal{E} \rightsquigarrow$ insieme di alberi di valutazione di espressioni e (stringa di soli terminali generati in Exp)
- $A \rightsquigarrow$ sono le espressioni aritmetiche
- $B \rightsquigarrow$ sono le espressioni booleane

Diamo semantica definendo un sistema di transizione

Stati : $\mathcal{E} \cup \mathcal{N}$ (stati terminali)

Relazione&Transizione: definita attraverso regole ccostruite induttivamente sulla grammatica.
Definiamo le seguenti regole sulle espressioni:

$$\mathcal{E}_1 : \overbrace{m \text{ op } n}^{\text{simboli}} \rightarrow p \text{ dove } p = \underbrace{m \text{ op } n}_{\text{calcolo effettivo}}$$

con $n, m \in \mathcal{N}$. Quindi ad esempio:

$$10 + 12 \rightarrow 22$$

Prendiamo le seguenti regole che impongono **una valutazione degli operandi left-to-right**:

$$\mathcal{E}_2 : \frac{e_0 \rightarrow e'_0}{e_o \text{ op } e_1 \rightarrow e'_0 \text{ op } e_1}$$

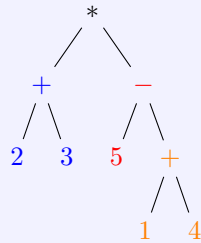
$$\mathcal{E}_3 : \frac{e_1 \rightarrow e'_1}{n \text{ op } e_1 \rightarrow m \text{ op } e'_1}$$

Esempio 5.1 (P)

endiamo per esempio questa espressione seguendo le regole che abbiamo appena definito:

$$\underbrace{(2 + 3)}_{e_0} * \underbrace{(5 - (1 + 4))}_{e_1}$$

Costruiamo l'albero di valutazione:



Le regole che abbiamo fissato non ci permettono di valutare prima e_1 e poi e_0 , ma ci obbliga a valutare prima e_0 e poi e_1 . Potremmo aggiungere questa regola alternativa per permettere la valutazione right-to-left:

$$\mathcal{E}'_3 : \frac{e_1 \rightarrow e'_1}{e_0 \text{ op } e_1 \rightarrow e_0 \text{ op } e'_1}$$

Vediamo ora le regole per le espressioni booleane:

$$B = \{true, false\} \text{ stati terminali per expr booleane}$$

$$\mathcal{E}_4 = t_1 \text{ or } t_2 \rightarrow t \text{ dove } t = \underbrace{t_1 \text{ or } t_2}_{\text{significato}}$$

con $t, t_1, t_2 \in B$. La regola $\mathcal{E}_2$ vale anche per le espressioni booleane.

$$\mathcal{E}'_3 = \frac{e_1 \rightarrow e'_1}{t \text{ or } e_1 \rightarrow t \text{ or } e'_1}$$

Uniamo $\mathcal{E}_3$ con $\mathcal{E}'_3$. Con $k \in \mathcal{N} \cup B$:

$$\mathcal{E}_3 = \frac{e_1 \rightarrow e'_1}{k \text{ bop } e_1 \rightarrow k \text{ bop } e'_1}$$

$$\mathcal{E}_5 = \frac{e \rightarrow e'}{\text{not } e \rightarrow \text{not } e'}$$

$$\mathcal{E}_6 = \text{not } t \rightarrow t \text{ dove } t' = \text{not } t$$

e $t, t' \in B$. Definiamo ora il sistema di transizione:

$$\langle \mathcal{E} \cup \mathcal{N} \cup B, \rightarrow \rangle$$

✚ Definizione 5.1: Valutazione

La funzione

$$Eval : Exp \rightarrow \mathcal{N} \cup B$$

di valutazione descrive il comportamento dinamico delle espressioni:

$$\forall e \in \mathcal{E}. Eval(e) = k \text{ sse } e \rightarrow^{*e} k$$

Dove $\rightarrow^{*e}$ è la chiusura transitiva di $\rightarrow$. quindi $\exists n \text{ t.c. } e \rightarrow e_1 \rightarrow e_2 \rightarrow \dots \rightarrow k$ con n transizioni.

✚ Definizione 5.2: Equivalenza di espressioni

$$\equiv \subseteq Exp \times Exp$$

è la relazione di equivalenza tra espressioni, definita come:

$$e_1 \equiv e_2 \text{ sse } Eval(e_1) = Eval(e_2)$$

5.1.1 Aspetti implementativi

L'ordine di valutazione degli operandi è un aspetto progettuale importante dove si vanno a definire:

- Operandi non definiti
- Side effects (modificano la memoria).
- Aritmetica finita

Per ragionare sull'ordine di valutazione dobbiamo capire come alcuni dei linguaggi di programmazione valutano le espressioni e ci sono due modi principali:

📌 Definizione 5.3: Valutazione eager e lazy

- **Eager**: valuta tutte le parti dell'espressione, quindi valuta anche quelle che non gli servono per restituire il risultato.
- **Lazy**: valuta solo le parti dell'espressione che gli servono per restituire il risultato.

📌 Esempio 5.2 (Operandi non definiti)

Un'espressione del tipo:

$$a == 0 ? b : b/a$$

Questa espressione può causare problemi perché se $a == 0$ allora b/a è non definita (divisione per zero). Dove in base al linguaggio di programmazione:

- Per la valutazione **eager**, valuta tutte le parti dell'espressione, quindi valuta anche b/a che è non definita (divisione per zero)
- Per la valutazione **lazy**, valuta la prima parte ma solo una volta che ha la risposta valuta quella che gli serve. In questo caso se $a == 0$ allora valuta b e restituisce il risultato, altrimenti valuta b/a e restituisce il risultato. Quindi non si ha una valutazione di un operando non definito.

Prendiamo il seguente esempio:

```
1 p := lista;
2 while (p != nil) and (p.valore != 3) do {...}
```

Se la valutazione è eager, questo codice mi dà un **errore** perché se mi valuta $p = nil$ ma valuta anche $p.valore$ e quindi mi dà errore. Con valutazione lazy, valuta $p = nil$ e restituisce false.

📌 Esempio 5.3 (Side Effects)

Prendiamo come esempio il seguente codice:

```
1 a = 10;
2 b := a + fun(&a); // fun modifica il parametro
```

Anche qua a seconda del linguaggio di programmazione e dell'ordine di valutazione degli operandi, si ha:

- Se valutiamo prima a allora a manterrà il valore 10.
- Altrimenti se valutiamo prima $fun(&a)$ allora a sarà modificato da fun e quindi b avrà un valore diverso.

5.2 Identificatori

Un identificatore è una sequenza di caratteri usata per riferirsi ad un altro oggetto (detto **denotabile**). Come ad esempio:

```
1 const pi := 3.14;  
2 int x;  
3 void f {...}
```

Dove chiaramente non posso usare come identificatori le keyword del linguaggio o altre specifiche che variano da linguaggio di programmazione. Li chiameremo "id".

$$\text{Id} = \{a, \text{rate}, b20, \dots\}$$

Utilizzeremo solitamente id o x .

5.2.1 Aspetti implementativi

Gli aspetti implementativi e progettuali che posso decidere se apportare al linguaggio e capire se voglio permettere:

- Nomi case-sensitive
- Parole riservate (come `const`, `var`, `procedure`, `if`, `while`, ecc...)
- Vincoli sulla lunghezza
- Presenza di caratteri speciali

5.2.2 Bindings

Consideriamo la seguente formula:

$$\left(\sum_{i+1}^n \left(\sum_{j+1}^m a_{i,j} \dots b_{j,k} \right) \right)$$

In questo caso, sto creando dei legami tra i miei identificatori i, j che sono sotto al raggio di definizione che li definisce e permette di dare significato. Queste due occorrenze vengono

chiamate **binding** ovvero creano il nome e il significato. Le occorrenze $a_{i,j}$ e $b_{j,k}$ vengono chiamate **occorrenze d'uso** o **applied**. In questo caso stiamo usando il nome per poter fare riferimento al suo significato. Tuttavia all'interno della formula ci sono occorrenze a cui non vengono date significato ovvero n, m, k che vengono chiamate **occorrenze libere** o **free**. Nei linguaggi di programmazione:

📌 Definizione 5.4: Bindings nei PL

Un identificatore è in posizione di definizione quando si attribuisce il significato all'identificatore.

📌 Definizione 5.5: Occorrenze d'uso nei PL

Un identificatore è in posizione di uso quando l'occorrenza serve per riferirsi al significato dell'identificatore (deve trovarsi nel raggio d'azione di una definizione).

📌 Definizione 5.6: Occorrenze libere nei PL

Un identificatore è in posizione libera se la sua occorrenza non si trova nel raggio d'azione di una definizione.

5.2.3 Tipi di bindings

Esistono diversi tipi di bindings a seconda dell'oggetto denotato:

- **Costante**: un identificatore che denota un valore costante, ad esempio π .
- **Variabile**: un identificatore che denota una cella di memoria che può essere modificata, ad esempio x .
- **Procedura**: un identificatore che denota un blocco di codice eseguibile, ad esempio f .

Questi legami sono **statici**. Alcuni legami possono cambiare, come ad esempio, locazione - valore.

- **Binding statico**: un legame che dura per tutta la durata del programma e non può essere cambiato.
- **Binding dinamico**: un legame che dura solo per un certo intervallo di tempo, ad esempio durante l'esecuzione di una procedura.

Nel momento in cui viene creato, viene definito il **tempo di binding**.

📌 Definizione 5.7: Ambiente

I legami vengono inseriti nell'**ambiente**, che non è altro che un insieme di bindings.

📌 Definizione 5.8: Termine Ground

Un termine è detto **ground** se non ha identificatori. Nel nostro caso, le espressioni che abbiamo definito.

📌 Definizione 5.9: Termine chiuso

Un termine è detto **chiuso** quando non ci sono identificatori in posizione libera.

Questi due termini hanno significato senza dover ricorrere ad un contesto esterno.

5.2.4 Ambienti in IMP

Aggiungendo ora i bindings al nostro linguaggio IMP, abbiamo che:

$\mathcal{E}$ espressioni

$N \cup B \rightsquigarrow$ sono anche valori denotabili

Creiamo ora l'ambiente all'interno di IMP:

$$Env = \bigcup_{I \subseteq_f Id} Env_I$$

dove $\subseteq_f$ è l'inclusione tra insiemi finiti.

$Env : I \rightarrow DVal$ dove ρ metavariable

è una funzione che associa un valore denotabile ad ogni identificatore in un insieme I finito.

✎ Esempio 5.4 (S)

pponiamo che ci venga dato questo ambiente:

$$I = \{x, y\}$$

$$\rho \in Env_I$$

$$\rho(x) = 2 \quad \rho(y) = 5$$

dove

$\rho \vdash x + y - 5$ è un espressione

Valutiamo $x + y - 5$ dentro ρ . La seguente è la grammatica definita dalle espressioni:

$$\begin{aligned} E &\rightarrow A \mid B \\ Exp &: A \rightarrow n \mid x \mid A \text{ op } A \\ B &\rightarrow true \mid false \mid x \mid A = A \mid not B \mid B \text{ or } B \end{aligned}$$

dove $x \in Id$. Espressioni da valutare in ambiente:

$$\rho \vdash e \rightarrow e'$$

Definiamo le regole della semantica: e viene valutata in e' dentro l'ambiente ρ .

- Regola 1:

$$\mathcal{E}_1 : \rho \vdash m \text{ op } n \rightarrow p \text{ dove } p = m \text{ op } n$$

- Regola 2:

$$\mathcal{E}_2 : \frac{\rho \vdash e_0 \rightarrow e'_0}{\rho \vdash e_0 \text{ op } e_1 \rightarrow e'_0 \text{ op } e_1}$$

- Regola 3:

$$\mathcal{E}_3 : \frac{\rho \vdash e_1 \rightarrow e'_1}{\rho \vdash k \text{ bop } e_1 \rightarrow k \text{ bop } e'_1}$$

- Regola 4:

$$\mathcal{E}_5 : \frac{\rho \vdash e \rightarrow e'}{\rho \vdash not e \rightarrow not e'}$$

- Regola 5:

$$\mathcal{E}_7 : \rho \vdash x \rightarrow k \text{ se } \rho(x) = k$$

$$x + y - 5 \rightsquigarrow \text{valutare in } \rho[x \mapsto 2, y \mapsto 5]$$

5.2.5 Scope

Consideriamo ancora la precedente formula

$$\underbrace{\left(\sum_{i+1}^n \underbrace{\left(\sum_{j+1}^m a_{i,j} \dots b_{j,k} \right)}_{\text{spazio di definizione di } j} \right)}_{\text{spazio di definizione di } i}$$

Se io dovessi usare j fuori da questo scope, esso assume un significato diverso o potrebbe essere anche libero. Stessa cosa chiaramente per i .

📌 Definizione 5.10: Scope

Lo **scope** è il raggio di azione di una definizione. Determina lo spazio in cui le potenziali occorrenze dell'identificatore definito fanno riferimento alla definizione stessa.

5.3 Tipi

Un **tipo** è l'insieme dei valori che una variabile può riferire. Tali valori devono condividere una certa proprietà invariante. L'utilizzo del tipo ha varie finalità:

- A tempo di progettazione, permette di organizzare delle funzionalità.
- A tempo di sviluppo serve a prevedere errori (controllo dimensionale).
- A tempo di implementazione, servono inoltre per capire quanta memoria allocare per una variabile.

Inoltre vi è un ulteriore legame, quello di tipo (aggiunto al legame con l'oggetto denotato).

5.3.1 Ambiente statico

L'**ambiente statico** è l'insieme di legami tra identificatore e tipi.

$$\underbrace{DTyp}_{\text{tipi denotabili}} = \{int, bool\} \quad [DVal = \mathcal{N} \cup B]$$

$$TEnv = \bigcup_{I \subseteq_f Id} TEnv_I$$

Dove $\subseteq_f$ è l'inclusione tra insiemi finiti di identificatori. L'insieme di tutti gli ambiente è l'insieme di tutte i possibili ambienti I finiti di identificatori.

$$TEnv = I \rightarrow DTyp$$

ad ogni identificatore in I associa un tipo in $DTyp$.

Metavariabile : $\Delta \in TEnv$ generico ambiente statico

$$\Delta \vdash_I e : \tau$$

asserisce che e è di tipo $\tau \in DTyp$ nell'ambiente statico Δ definito sugli identificatori in I . Dove $\vdash_I$ è la relazione di giudizio di tipo, che è definita attraverso regole costruite induttivamente sulla grammatica delle espressioni.

Se le regole di semantica statica permettono di associare un tipo ad una espressione allora diciamo che l'espressione è **ben formata**.

5.3.2 Semantica statica

Quello che andremo a definire è un insieme di regole che permettono di verificare la corretta costruzione (in termini di tipi) delle espressioni. Questo sistema lo chiameremo **semantica statica**.

- Assioma 1:

$$\mathcal{E}_{s_1} : \vdash n : int$$

- Assioma 2:

$$\mathcal{E}_{s_2} : \vdash t : bool$$

- Assioma 3:

$$\mathcal{E}_{s_3} : \Delta \vdash_I x : \tau \text{ se } x \in I, \Delta(x) = \tau$$

- Per le espressioni aritmetiche:

$$\mathcal{E}_{s_4} : \frac{\Delta \vdash_I e_1 : int \quad \Delta \vdash_I e_2 : int}{\Delta \vdash_I e_1 \text{ op } e_2 : int}$$

$$\mathcal{E}_{s_5} : \frac{\Delta \vdash_I e_1 : bool \quad \Delta \vdash_I e_2 : bool}{\Delta \vdash_I e_1 \text{ op } e_2 : int}$$

- Per le espressioni booleane:

$$\mathcal{E}_{s_6} : \frac{\Delta \vdash_I e : bool}{\Delta \vdash_I \text{ not } e : bool}$$

$$\mathcal{E}_{s_7} : \frac{\Delta \vdash_I e_1 : int \quad \Delta \vdash_I e_2 : int}{\Delta \vdash_I e_1 = e_2 : bool}$$

5.3.3 Ambienti compatibili

Definizione 5.11: Ambiente compatibile

Dato un ambiente dinamico che associa valori a nomi agli identificatori I , $\rho \in Env_I$ e $\Delta \in TEnv_I$ un ambiente statico che associa tipi a nomi agli identificatori I , diciamo che ρ e Δ sono compatibili se

$$\forall Id \in I . \Delta(Id) = \tau \text{ sse } \rho(Id) = \tau \text{ (è un valore di tipo } \tau \text{)}$$

Esempio 5.5

Dati:

$$\Delta = [x \mapsto int, y \mapsto bool]$$

$$\rho = [x \mapsto 2, y \mapsto true]$$

NON sono compatibili.

Se usiamo la notazione $\rho \vdash_{\Delta}$ per indicare che ρ e Δ sono compatibili. Abbiamo bisogno per definire gli ambienti (sia Δ che ρ) di una categoria sintattica che permette di creare e modificare legami che vanno a costituire l'ambiente.

5.4 Dichiarazioni

Le **dichiarazioni** sono una categoria sintattica che fornisce il meccanismo (implicito o esplicito) per creare legami (tra identificatori e tipi (statico) o oggetti (dinamico)). Inoltre denotano richieste di modifica o creazione di ambienti. Le dichiarazioni vengono elaborate per attuare richieste di modifica o creazione di ambienti.

5.4.1 Sintassi

Definiamo la sintassi delle dichiarazioni:

$$\mathcal{D} : \quad D \rightarrow nil \mid const\ x : \tau = e \mid D; D \mid D\ in\ D \mid \rho$$

dove

- nil è la dichiarazione nulla che non crea nessun legame.
- $const\ x : \tau = e$ è la dichiarazione di una costante x di tipo τ e valore rappresentato da e .
- $D; D$ è la composizione sequenziale di dichiarazioni, che permette di creare legami multipli.
- $D\ in\ D$ è la composizione di dichiarazioni con annidamento, che permette di creare legami con scope annidati.
- ρ è una dichiarazione che rappresenta un ambiente dinamico

5.4.2 Aggiornamento di un ambiente

Consideriamo β, β' ambienti. Cerchiamo di definire la seguente operazione:

$$\beta[\beta'] \quad \beta \text{ modificato da } \beta'$$

$$\beta : I \quad I \subseteq_f Id(\beta \text{ è definito su } I)$$

$$\beta' : I' \quad I' \subseteq_f Id(\beta' \text{ è definito su } I')$$

Dove l'aggiornamento di un ambiente:

$$\beta[\beta'] = \bar{\beta} \text{ nuovo ambiente } \bar{\beta} : I \cup I'$$

$$\forall x \in I' . \bar{\beta}(x) = \beta'(x)$$

$$\forall x \in I \setminus I' . \bar{\beta}(x) = \beta(x)$$

Intuitivamente quindi $\bar{\beta}$ è un nuovo ambiente in cui prendo tutti i legami di β e β' e per i legami in comune tengo solo β' .

Esempio 5.6

Prendiamo per esempio l'ambiente dinamico:

$$\rho = [x \mapsto 2, y \mapsto true, z \mapsto 5]$$

$$\rho' = [x \mapsto 5, w \mapsto false]$$

dove

$$\rho[\rho'] = [x \mapsto 5, y \mapsto true, z \mapsto 5, w \mapsto false]$$

Come possiamo notare, abbiamo sovrascritto x con il nuovo legame (5), abbiamo mantenuto y e z e abbiamo aggiunto w .

5.4.3 Composizione sequenziale

Prendiamo per esempio D_1 e D_2 due dichiarazioni.

Ogni dichiarazione successiva o aggiunge nuovi legami o sovrascrive legami esistenti. L'ambiente risultante è così visibile al codice successivo.

5.4.4 Composizione privata

I legami creati o sovrascritti da D_1 sono ad uso esclusivo dell'elaborazione di D_2 e non visibili al codice successivo.

Esempio 5.7

```
1 const x : int = 3;
2 const x : int = 5 in y : int = 6 * x;
3 const z : int = x * y;
```

- Nella prima dichiarazione, x è legato a 3.

$$\rho = [x \mapsto 3]$$

- Nella seconda dichiarazione, associamo x a 5 ma solo per la dichiarazione $y : int =$

$6 * x$. Quindi y è legato a $6 * 5 = 30$.

$$\rho[x \mapsto 5] = [x \mapsto 5] = \rho_1$$

dove

$$\rho_2 = [y \mapsto 6 * 5 = 30]$$

- Nella terza dichiarazione z è legato a $x * y$.

$$\rho[\rho_2] = [x \mapsto 3, y \mapsto 30]$$

Esempio 5.8

```
1 const x : int = 3;  
2 const x : int = 5;  
3 const y : int = 6 * x;  
4 const z : int = x * y;
```

- Nella prima dichiarazione, x è legato a 3.
- Nella seconda dichiarazione, associamo x a 5 e sovrascriviamo il legame precedente.
- Nella terza dichiarazione y è legato a $6 * 5 = 30$.
- Nella quarta dichiarazione z è legato a $5 * 30 = 150$.

5.4.5 Identificatori in posizione libera e di definizione

Caratterizziamo gli identificatori in posizione libera (FI) e in posizione di definizione (DI).

$$FI(Exp \cup Dec) = I \subseteq Id \quad FI : Exp \cup Dec \rightarrow \mathcal{P}(Id)$$

Ricordiamo come gli identificatori **liberi** non hanno un legame, quindi richiedono un contesto esterno per avere significato. Definiamo FI per induzione sulle espressioni:

- $FI(k) = \emptyset$ con $k \in \mathcal{N} \cup B$.
- $FI(x) = \{x\}$ perché x è in posizione di uso e quindi è libero.
- $FI(e_1 \text{ op } e_2) = FI(e_1) \cup FI(e_2)$ perché se e_1 o e_2 hanno identificatori in posizione libera, allora anche $e_1 \text{ op } e_2$ li ha.
- $FI(\text{not } e) = FI(e)$ perché se e ha identificatori in posizione libera, allora anche $\text{not } e$ li ha.

Per le dichiarazioni:

- $FI(nil) = \emptyset$
- $FI(constx : \tau = e) = FI(e)$ perché x è in posizione di definizione e quindi non è libero.
- $FI(D_1; D_2) = FI(D_1) \cup FI(D_2 \setminus DI(D_1))$
- $FI(D_1 \text{ in } D_2) = (FI(D_1) \cup FI(D_2)) \setminus DI(D_1)$

Definiamo ora DI , che restituisce gli identificatori che nel costrutto vengono legati a un significato che è visibile all'esterno del costrutto.

$$DI : Exp \cup Dec \rightarrow \mathcal{P}(Id)$$

Per le espressioni, sappiamo che non ci sono identificatori in posizione di definizione, quindi:

$$DI(e) = \emptyset \text{ con } \forall e \in Exp$$

Per le dichiarazioni:

- $D(nil) = \emptyset$
- $D(constx : \tau = e) = \{x\}$
- $D(\rho) = I$ se $\rho \in Env_I$
- $D(D_1; D_2) = D(D_1) \cup D(D_2)$ tutti gli id legati in D_1 e D_2 restano poi legati al loro significato.
- $D(D_1 \text{ in } D_2) = D(D_2)$ solo gli id legati in D_2 hanno significato dopo

5.4.6 Analisi semantica per le dichiarazioni

L'analisi semantica per le dichiarazioni associa un ambiente statico alle dichiarazioni ben formate. Dato Δ è il contesto, con eventuali dichiarazioni precedenti, questa è la seguente forma delle regole:

$$\Delta \vdash d : \Delta' \text{ con } \Delta' \text{ l'ambiente definiti e quindi associato da } d$$

- Assioma 1:

$$\mathcal{D}_{S1} : \vdash nil : \emptyset$$

- Assioma 2:

$$\mathcal{D}_{S2} : \vdash \rho : \Delta$$

dove ρ e Δ sono compatibili.

- Assioma 3:

$$\mathcal{D}_{S3} : \frac{\Delta \vdash e : \tau}{\Delta \vdash \text{const } x : \tau = e : [x \mapsto \tau]}$$

Dove τ è una metavariable per lo stesso oggetto. Quindi devono essere tutti uguali. Se dichiaro x di tipo *int*, l'espressione deve essere intera e se dichiaro x booleana l'espressione deve essere booleana.

- Assioma 4:

$$\mathcal{D}_{S4} : \frac{\Delta \vdash d_1 : \Delta_1 \quad \Delta[\Delta_1] \vdash d_2 : \Delta_2}{\Delta \vdash d_1 \text{ in } d_2 : \Delta_2}$$

- Assioma 5:

$$\mathcal{D}_{S5} : \frac{\Delta \vdash d_1 : \Delta_1 \quad \Delta[\Delta_1] \vdash d_2 : \Delta_2}{\Delta \vdash d_1; d_2 : \Delta_1[\Delta_2]}$$

5.4.7 Semantica dinamica

L'ambiente statico generato dalla semantica statica è compatibile con l'ambiente dinamico:

$$\rho \vdash_{\Delta} d \rightarrow_d d' \text{ forma delle regole}$$

d viene elaborato (1 passo) in d' nell'ambiente dinamico ρ . Gli stati sono le dichiarazioni e gli stati terminali sono gli ambienti:

$$\text{Stati} : \mathcal{D} \cup \text{Env}$$

Definiamo ora le regole:

- Assioma 1:

$$\mathcal{D}_1 : \vdash \text{nil} \rightarrow \emptyset \text{ perché } d = \rho \text{ è già terminale non abbiamo nulla da elaborare}$$

- Assioma 2:

$$\mathcal{D}_2 : \rho \vdash \text{const } x : \tau = k \rightarrow [x \mapsto k]$$

- Assioma 3:

$$\mathcal{D}_3 : \frac{\rho \vdash e \rightarrow_e e'}{\rho \vdash \text{cost } x : \tau = e \rightarrow_d \text{const } x : \tau = e'}$$

- Sequenziale:

$$\mathcal{D}_4 : \frac{\rho \vdash d_1 \rightarrow_d d'_1}{\rho \vdash d_1 ; d_2 \rightarrow d'_1 ; d_2}$$

$$\mathcal{D}_5 : \frac{\rho[\rho_1] \vdash d_2 \rightarrow_d d'_2}{\rho \vdash \rho_1 ; d_2 \rightarrow \rho_1 ; d_2}$$

$$\mathcal{D}_6 : \rho \vdash \rho_1 ; \rho_2 \rightarrow_d \rho_1[\rho_2]$$

- Privata:

$$\mathcal{D}_7 : \frac{\rho \vdash d_1 \rightarrow_d d'_1}{\rho \vdash d_1 \text{ in } d_2 \rightarrow_d d'_1 \text{ in } d_2}$$

$$\mathcal{D}_8 : \frac{\rho[\rho_1] \vdash d_2 \rightarrow_d d'_2}{\rho \vdash \text{rho}_1 \text{ in } d_2 \rightarrow_d \rho_1 \text{ in } d'_2}$$

$$\mathcal{D}_9 : \rho \vdash \rho_1 \text{ in } \rho_2 \rightarrow_d \rho_2$$

Definizione 5.12: Funzione di elaborazione

La funzione di elaborazione prende una dichiarazione e restituisce un ambiente, ovvero l'ambiente che viene creato o modificato da quella dichiarazione.

$$Elab : \mathcal{D} \rightarrow Env$$

dove $\mathcal{D}$ è generato dalla grammatica delle dichiarazioni

$$\forall d \in \mathcal{D}. Elab(d) = \rho \text{ sse } \vdash d \rightarrow_d^* \rho$$

Teorema 5.4.1 Equivalenza di dichiarazioni

$\equiv \subseteq \mathcal{D} \times \mathcal{D}$ è la relazione di equivalenza tra dichiarazioni

dove

$$\forall d_1, d_2 \in \mathcal{D}. d_1 \equiv d_2 \text{ sse } Elab(d_1) = Elab(d_2)$$

Esempio 5.9

```
1 const x : int = 2 in const y : int = x + 1; const z : int = x + y;
```

Prendiamo:

- `const x : int = 2` è d_1
- `const y : int = x + 1; const z : int = x + y` è d_2

I passi che dobbiamo fare ora sono:

- Dobbiamo elaborare d_1 in d_2 :

$$\frac{\vdash d_1 \rightarrow_d d'_1 \quad (\text{oppure } p)}{\vdash d_1 \text{ in } d_2 \rightarrow_d d'_1 \text{ in } d_2}$$

nel nostro caso:

$$\frac{\vdash \text{const } x : \text{int} = 2 \rightarrow_d [x \mapsto 2]}{\vdash d_1 \text{ in } d_2 \rightarrow_d [x \mapsto 2] \text{ in } d_2}$$

Regola ora da applicare:

$$\frac{\rho_1 \vdash d_2 \rightarrow d'_2}{\vdash \rho_1 \text{ in } d_2 \rightarrow \rho_1 \text{ in } d'_2} \equiv \frac{\rho_1 \vdash d_2 \rightarrow_d^* \rho_2}{\vdash \rho_1 \text{ in } d_2 \rightarrow \rho_1 \text{ in } \rho_2}$$

- Ora dobbiamo elaborare d_2 che può essere scomposto in d_3 e d_4 : Per elaborare d_3 dobbiamo usare la seguente regola:

$$\frac{\rho_1 \vdash d_3 \rightarrow_d \rho'_3}{\rho_1 \vdash \text{const } y : \text{int} = x + 1 \rightarrow_d \rho'_3}$$

$$\frac{\rho_1 \vdash x + 1 \rightarrow_d^* ?}{\rho_1 \vdash \text{const } y : \text{int} = x + 1 \rightarrow_d \text{const } y : \text{int} = ?}$$

dove sappiamo "unfoldare" il contesto:

$$\frac{\rho_1 \vdash x \rightarrow 2}{\rho_1 \vdash x + 1 \rightarrow 2 + 1}$$

di conseguenza riscriviamo la regola:

$$\frac{\rho_1 \vdash x + 1 \rightarrow_d^* 3}{\rho_1 \vdash \text{const } y : \text{int} = x + 1 \rightarrow_d \text{const } y : \text{int} = 3}$$

quindi $\rho \vdash \text{const } y : \text{int} = 3 \rightarrow [y \mapsto 3]$. Riscriviamo la regola:

$$\frac{\rho_1 \vdash d_3 \rightarrow_d [y \mapsto 3]}{\rho_1 \vdash d_3 ; d_4 \rightarrow \rho_3 ; d_4}$$

- Ora preoccupiamoci di d_4 . La regola che dobbiamo utilizzare è la seguente:

$$\frac{\rho_1[\rho_3] \rightarrow_d \rho_4}{\rho_1 \vdash \rho_3 ; d_4 \rightarrow \rho_3 ; \rho_4}$$

Elaboriamo d_4 , $d_4 = \text{const}; z : \text{int} = x + y$ in $\rho_1[\rho_3] = [x \mapsto 2, y \mapsto 3]$: Le regole da

applicare sono:

$$\frac{\rho_1[\rho_3] \vdash x + y \rightarrow_d^* k}{\rho_1[\rho_3] \vdash \text{const } z : \text{int} = x + y \rightarrow_d \text{const } z : \text{int} = k}$$

Sviluppiamo $x + y$:

$$\frac{\rho_1[\rho_3] \vdash x \rightarrow 2}{\rho_1[\rho_3] \vdash x + y \rightarrow 2 + y}$$

$$\frac{\rho_1[\rho_3] \vdash y \rightarrow 3}{\rho_1[\rho_3] \vdash 2 + y \rightarrow 2 + 3}$$

$$\rho_1[\rho_3] \vdash 2 + 3 \rightarrow 5$$

Riscrivo la regola istanziata:

$$\frac{\rho_1[\rho_3] \vdash x + y \rightarrow_d^* 5}{\rho_1[\rho_3] \vdash \text{const } z : \text{int} = x + y \rightarrow_d \text{const } z : \text{int} = 5}$$

Applichiamo l'assioma:

$$\rho_1[\rho_3] \vdash \text{const } z : \text{int} = 5 \rightarrow [z \mapsto 5]$$

- Torniamo all'elaborazione di $d_2 = d_3 ; d_4$: Descriviamo l'ultima regola usata per d_2 :

$$\frac{\rho_1[\rho_3] \vdash d_4 \rightarrow^* \rho_4}{\rho_1 \vdash d_3 ; d_4 \rightarrow_d \rho_3 ; \rho_4}$$

Applichiamo l'assioma:

$$\rho_1 \vdash \rho_3 ; \rho_4 \rightarrow_d \rho_3[\rho_4] = \rho_2 = [y \mapsto 3, z \mapsto 5]$$

Riscriviamo la regola:

$$\frac{\rho_1 \vdash d_2 \rightarrow^* \rho_2}{\vdash \rho_1 \text{ in } d_2 \rightarrow \rho_1 \text{ in } \rho_2}$$

Applichiamo l'assioma:

$$\vdash \rho_1 \text{ in } \rho_2 \rightarrow_d \rho_2 \Rightarrow \vdash d_1 \text{ in } d_2 \rightarrow^* \rho_2 = [y \mapsto 3, z \mapsto 5]$$

5.5 Variabili

L'introduzione di **identificatori variabili** richiede l'introduzione della **memoria** e dei **comandi**.

$$x := x + 1$$

dove

- x è il target e lo scriviamo $l - value$
- $x + 1$ è l'espressione da valutare e lo scriviamo $r - value$

Ci sono alcuni problemi che l'ambiente non ci permette di distinguere:

- Gestire diversi oggetti con lo stesso nome (ricorsione, ridefinizione di variabili...).
- Gestire diversi nomi per lo stesso oggetto (puntatori, aliasing...).
- Nomi e valori che cambiano durante l'esecuzione.

Dobbiamo introdurre il concetto della **locazione**.

5.5.1 Locazione

Una **locazione** è un identificatore che denota una cella di memoria, ovvero un oggetto che può essere modificato durante l'esecuzione. Quindi sta "in mezzo" tra l'identificatore e l'oggetto denotato, e permette di distinguere tra oggetti con lo stesso nome e nomi diversi per lo stesso oggetto.



📌 Definizione 5.13: Memoria

La **memoria** tiene traccia delle trasformazioni effettuate dall'esecuzione di programmi.

📌 Definizione 5.14: Locazione

La **locazione** è un'astrazione che viene creata dalle dichiarazioni:

- Se la creazione è implicita, viene creata dai comandi a tempo di esecuzione (nei linguaggi dinamici/interpretati). Altrimenti se è esplicita, allora viene creata a tempo di compilazione.
- Ha un tempo di vita
- Viene usato per contenere/riferire a oggetti che possono cambiare durante l'esecuzione
- sono considerate illimitate, quindi non dobbiamo preoccuparci di esaurirle.

In una locazione possiamo inserire:

- Gli **identificatori costanti** non usano la memoria poiché vengono associati a valori che non cambiano durante l'esecuzione. Sono legami che restano nell'ambiente.

$$x \rightarrow 2 \quad x \text{ sarà sempre } 2$$

Useremo la notazione **const**.

- Le **variabili** usano identificate da due legami: uno nell'ambiente:

$$x \rightarrow \square \text{ (locazione)}$$

e uno nella memoria:

$$\square \rightarrow 2 \text{ (valore)}$$

il cui valore memorizzabile può essere modificato durante l'esecuzione. Useremo la notazione **var**.

All'interno di IMP abbiamo:

- Valori esprimibili (elementi primitivi di Exp, come i booleani e gli interi)
- Valori memorizzabili ($B \cup \mathcal{N}$) ovvero che possono essere associati ad una locazione.
- Valori denotabili ($B \cup \mathcal{N} \cup \underbrace{L}_{\text{locazioni}}$) ovvero che possono essere legati da un identificatore.

$\mathcal{D}$ viene modificata aggiungendo la dichiarazione di variabile.

$$\mathcal{D} : D \rightarrow nil \mid \text{const } x : \tau = e \mid \text{var } x : \tau \mid D; D \mid D \text{ in } D \mid \rho \mid \text{var } x : \tau = e$$

$var\ x : \tau = e$ è la dichiarazione di una variabile x di tipo τ che viene inizializzata con un valore e .

$$\begin{cases} FI(var\ x : \tau = e) = FI(e) \\ DI(var\ x : \tau = e) = \{x\} \end{cases}$$

Dobbiamo aggiungere i tipi $DTyp$ per le locazioni.

$$DTyp = \{int, bool, \underbrace{intloc, boolloc}_{\text{tipi delle locazioni } (\tau\ loc)}\}$$

Regola statica per var:

$$\frac{\Delta \vdash e : \tau}{\Delta \vdash var\ x : int \rightarrow [x \mapsto \tau loc]}$$

Ovvero che x è una variabile di tipo τ se e è un'espressione di tipo τ .

5.5.2 Semantica statica per le dichiarazioni con variabili

La semantica statica verifica che la dichiarazione sia ben formata associando un ambiente statico alla dichiarazione. Bisogna verificare, a livello statico, che non sia possibile modificare il valore di una costante. Per fare ciò bisogna aggiungere i tipi per le locazioni:

$$DTyp = \{int, bool, intloc, boolloc\}$$

Aggiungiamo una nuova regola alla semantica statica delle dichiarazioni per gestire le variabili:

- Regola 6:

$$\frac{\Delta \vdash e : \tau}{\Delta \vdash \text{var } x : \tau = e : [x \mapsto \tau loc]}$$

Definizione 5.15

Anche nelle espressioni gli identificatori possono essere variabili e questo non cambia la sintassi.

Ricordiamo l'assioma per gli identificatori

$$\Delta \vdash x : \tau \quad \text{se } \Delta(x) = \tau$$

Dove τ è il tipo associato a x . Ora che abbiamo aggiunto le locazioni nei tipi, bisogna gestire anche i tipi delle locazioni e la nuova regola per gli identificatori diventa:

$$\Delta \vdash x : \tau \quad \text{se } \Delta(x) \in \{\tau, \tau loc\}$$

Quindi i valori a cui x si riferisce sono sempre di tipo τ , ma la dichiarazione di x può essere di **costante** ($\Delta(x) = \tau$) o di **variabile** ($\Delta(x) = \tau\text{loc}$).

5.5.3 Aggiornamento della memoria

Bisogna definire la **memoria** e integrarla nelle regole della semantica dinamica per le dichiarazioni. Consideriamo:

- Loc l'insieme delle locazioni
- $new(L)$ è la funzione che restituisce una nuova, cioè che non appartiene a L , locazione
- $Mem_L : L \rightarrow \mathcal{V}$ è una funzione che associa ad ogni locazione un valore
- $Mem = \bigcup_{L \subseteq_f Loc} Mem_L$ è l'insieme di tutte le funzioni di memoria.
- $Env_I : I \rightarrow \mathcal{V} + Loc$ è una funzione che associa ad ogni identificatore un valore o una locazione
- $Env = \bigcup_{I \subseteq_f Id} Env_I$ è l'insieme di tutte le funzioni di ambiente

Siano $\sigma \in Env_I$ e $\sigma' \in Env_{I'}$, l'aggiornamento della memoria si rappresenta come:

$$\sigma[\sigma']$$

dove σ è modificata da σ' e genera una nuova memoria σ'' tale che:

$$\begin{aligned} \forall x \in I' . \sigma''(x) &= \sigma'(x) \\ \forall x \in I \setminus I' . \sigma''(x) &= \sigma(x) \end{aligned}$$

cioè σ'' è una memoria che contiene tutte le associazioni di σ' e tutte le associazioni di σ che non sono sovrascritte da σ' :

$$\sigma'' \in Env_{I \cup I'}$$

5.5.4 Semantica dinamica per le espressioni con identificatori variabili

Consideriamo:

- **Stati:** $\mathcal{E} \times Mem$. Gli stati intermedi sono l'espressione da valutare e la memoria iniziale
- **Stati terminali:** $\mathcal{V} \times Mem$ cioè il valore associato all'espressione più la memoria

La forma della regola sarà del tipo:

$$\rho \vdash \langle e, \sigma \rangle \rightarrow_e \langle e', \sigma \rangle$$

Seguendo questa forma possiamo aggiornare gli assiomi per le espressioni in modo da gestire le variabili:

-

$$\rho \vdash \langle m \text{ op } n, \sigma \rangle \rightarrow \langle p, \sigma \rangle \quad p = m \text{ op } n$$

-

$$\rho \vdash \langle t_1 \text{ or } t_2, \sigma \rangle \rightarrow \langle t, \sigma \rangle \quad t = t_1 \text{ or } t_2$$

-

$$\rho \vdash \langle \text{not } t, \sigma \rangle \rightarrow \langle t', \sigma \rangle \quad t' = \text{not } t$$

Quindi si ha che:

$$\rho \vdash \langle x, \sigma \rangle \rightarrow \langle k, \sigma \rangle$$

- Se $\rho(x) = l \in Loc$ allora $\sigma(l) = k$, cioè x è variabile
- Altrimenti $\rho(x) = k$ cioè x è costante

Ora bisogna aggiornare anche le altre regole per gestire le variabili:

-

$$\frac{\rho \vdash \langle e_1, \sigma \rangle \rightarrow_e^* \langle k, \sigma \rangle}{\rho \vdash \langle e_1 \text{ bop } e_2, \sigma \rangle \rightarrow \langle k \text{ bop } e_2, \sigma \rangle}$$

-

$$\frac{\rho \vdash \langle e_2, \sigma \rangle \rightarrow_e^* \langle k', \sigma \rangle}{\rho \vdash \langle e_2 \text{ bop } e_1, \sigma \rangle \rightarrow \langle k \text{ bop } k', \sigma \rangle}$$

-

$$\frac{\rho \vdash \langle e, \sigma \rangle \rightarrow_e^* \langle t, \sigma \rangle}{\rho \vdash \langle \text{not } e, \sigma \rangle \rightarrow \langle \text{not } t, \sigma \rangle}$$

- **Valutazione:** La valutazione per gestire le variabili diventa:

$$Eval : Exp \times Mem \rightarrow \mathcal{V}$$

dove:

$$Eval(e, \sigma) = k \iff \vdash \langle e, \sigma \rangle \rightarrow^* k$$

- **Equivalenza:** L'equivalenza per gestire le variabili diventa:

$$\equiv \subseteq Exp \times Exp$$

dove:

$$e_1 \equiv e_2 \iff \forall \sigma \in Mem . Eval(e_1, \sigma) = Eval(e_2, \sigma)$$

Cioè qualunque sia il valore associato agli identificatori la valutazione delle espressioni deve restare la stessa

5.5.5 Semantica dinamica per le dichiarazioni con variabili

- **Stati:** $Dec \times Mem$
- **Stati terminali:** $Env \times Mem$

La forma della regola sarà del tipo:

$$\vdash \langle d, \sigma \rangle \rightarrow_d \langle d', \sigma' \rangle$$

L'elaborazione porta ad una memoria σ' perchè la memoria σ può essere stata modificata siccome le dichiarazioni di variabile **inizializzano gli identificatori anche nella memoria**.

Usando questa forma possiamo aggiornare gli assiomi per le dichiarazioni in modo da gestire le variabili:

•

$$\rho \vdash \langle \text{var } x: \tau = k, \sigma \rangle \rightarrow \langle [x \mapsto l], \sigma[l \mapsto k] \rangle$$

dove $l \in Loc$ è una nuova locazione per x e la memoria σ viene aggiornata con l'associazione $l \mapsto k$ per inizializzare la variabile x con il valore k .

Bisogna aggiornare anche le regole della dichiarazione composta per gestire le variabili (semplicemente propagando la memoria):

•

$$\rho \vdash \langle \text{const } x: \tau = k, \sigma \rangle \rightarrow_d \langle [x \mapsto k], \sigma \rangle$$

•

$$\rho \vdash \langle \text{nil}, \sigma \rangle \rightarrow_d \langle \emptyset, \sigma \rangle$$

•

$$\frac{\rho \vdash \langle d_1, \sigma \rangle \rightarrow_d^* \langle \rho_1, \sigma' \rangle}{\rho \vdash \langle d_1; d_2, \sigma \rangle \rightarrow_d \langle \rho_1; d_2, \sigma' \rangle}$$

•

$$\frac{\rho[\rho_1] \vdash \langle d_2, \sigma \rangle \rightarrow_d^* \langle \rho_2, \sigma' \rangle}{\rho \vdash \langle \rho_1; d_2, \sigma \rangle \rightarrow_d \langle \rho_1; \rho_2, \sigma' \rangle}$$

•

$$\rho \vdash \langle \rho_1; \rho_2, \sigma \rangle \rightarrow_d \langle \rho_1[\rho_2], \sigma \rangle$$

E anche le regole per la dichiarazione privata:

•

$$\frac{\rho \vdash \langle d_1, \sigma \rangle \rightarrow_d^* \langle \rho_1, \sigma' \rangle}{\rho \vdash \langle d_1 \text{ in } d_2, \sigma \rangle \rightarrow_d \langle \rho_1 \text{ in } d_2, \sigma' \rangle}$$

•

$$\frac{\rho[\rho_1] \vdash \langle d_2, \sigma' \rangle \rightarrow_d^* \langle \rho_2, \sigma' \rangle}{\rho \vdash \langle \rho_1 \text{ in } d_2, \sigma' \rangle \rightarrow_d \langle \rho_1 \text{ in } \rho_2, \sigma' \rangle}$$

•

$$\rho \vdash \langle \rho_1 \text{ in } \rho_2, \sigma \rangle \rightarrow_d \langle \rho_1[\rho_2], \sigma \rangle$$

- **Elaborazione:** L'elaborazione per gestire le variabili diventa:

$$Elab : Dec \times Mem \rightarrow Env \quad \underbrace{\times Mem}_{\text{Se vogliamo considerare anche i side effects}}$$

dove:

$$\begin{aligned} \forall d \in Dec, \sigma \in Mem . Elab(d, \sigma) &= (\rho, \sigma') \\ \iff \exists \sigma' . \vdash \langle d, \sigma \rangle \rightarrow_d^* \langle \rho, \sigma' \rangle \end{aligned}$$

Oppure $\not\vdash \sigma'$ se si vogliono considerare anche i side effects

- **Equivalenza:** L'equivalenza per gestire le variabili diventa:

$$\equiv \subseteq Dec \times Dec$$

dove:

$$\forall d_1, d_2 . d_1 \equiv d_2 \iff \forall \sigma Elab(d_1, \sigma) = Elab(d_2, \sigma)$$

5.6 Comandi

I comandi sono una categoria sintattica il cui ruolo principale consiste nella trasformazione (irreversibile) della memoria. I comandi vengono **eseguiti** per attuare le richieste di trasformazione della memoria che rappresentano.

📌 Definizione 5.16: Reversabilità

Non esiste alcun meccanismo automatico che annulla la trasformazione

Esistono due tipi di comandi

1. **Comandi base:** sono i comandi che denotano le trasformazioni della memoria. Ad esempio, l'assegnamento o il comando nullo
2. **Comandi di controllo del flusso:** sono i comandi che determinano un controllo del flusso di esecuzione. Ad esempio i cicli o le istruzioni condizionali

5.6.1 Sintassi dei comandi

Aggiungiamo i comandi alla sintassi del linguaggio:

$$Com : C \rightarrow \overbrace{\text{skip} \mid x := e}^{\text{Comandi base}} \mid C; C \mid \text{while } B \text{ do } C \mid \text{if } B \text{ then } C \text{ else } C$$

- **Assegnamento:** modifica il valore della variabile x con la valutazione dell'espressione e
- **Variabile:** la variabile ha le seguenti caratteristiche:
 - Nome
 - Tipo
 - Valore memorizzato (ciò che è salvato in memoria)
 - Locazione (la posizione in memoria a cui la variabile fa riferimento)
 - **Scope** (l'area di codice del programma in cui la variabile è visibile e accessibile)
 - **Lifetime o tempo di vita** (l'intervallo di tempo durante il quale la variabile esiste e può essere utilizzata)

5.6.2 Semantica statica dei comandi

La semantica statica dei comandi verifica che i comandi siano ben formati e quindi fa solo **controlli di coerenza di tipi**. Consideriamo l'ambiente statico Δ e definiamo la semantica statica per i comandi:

- $FI(x := e) = FI(e) \cup \{x\}$. Per far sì che l'assegnamento abbia significato, tutti devono avere un significato, quindi x deve essere definito e e deve essere ben formata
- $DI(x := e) = \emptyset$. L'assegnamento non definisce nuovi identificatori, ma modifica il valore di un identificatore **già definito**

•

$$\frac{\Delta \vdash e : \tau}{\Delta \vdash x := e} \Delta(x) = \tau_{\text{loc}}$$

$\Delta(x)$ deve essere di tipo locazione τ_{loc} perchè x è una variabile e quindi può essere usata come L-value in un assegnamento. Quindi si ha che $x := e$ è ben formata se:

1. x è di tipo locazione
 2. x ed e hanno lo stesso tipo di valore τ
- $\Delta \vdash \text{skip}$ non ha vincoli da verificare

5.6.3 Semantica dinamica dei comandi

La semantica dinamica dei comandi descrive come i comandi modificano la memoria durante l'esecuzione. Consideriamo:

- **Stati:** $Com \times Mem$
- **Stati terminali:** Mem , cioè i comandi vengono eseguiti per manipolare la memoria

La forma della regola sarà del tipo:

$$\rho \vdash \langle c, \sigma \rangle \rightarrow_c \langle c', \sigma' \rangle$$

Usando questa forma possiamo definire la semantica dinamica per i comandi:

- Skip

$$\rho \vdash \langle \mathbf{skip}, \sigma \rangle \rightarrow_c \sigma$$

- Assegnamento: si valuta l'espressione e in k con un certo numero di passi partendo da σ

$$\frac{\rho \vdash \langle e, \sigma \rangle \rightarrow_e^* \langle k, \sigma' \rangle}{\rho \vdash \langle \mathbf{x} := e, \sigma \rangle \rightarrow \langle \mathbf{x} := k, \sigma' \rangle}$$

- Assioma per l'assegnamento:

$$\rho \vdash \langle \mathbf{x} := k, \sigma \rangle \rightarrow_c \sigma[l \mapsto k], \rho(x) = l$$

dove ρ indica quale locazione è legata ad x

Esempio 5.10

n esempio di esecuzione del seguente assegnamento:

$$\mathbf{x} := \mathbf{x} * \mathbf{y}$$

x e y sono **liberi** e quindi per eseguire l'assegnamento abbiamo bisogno degli ambienti contestuali e poi dobbiamo usare una memoria. Supponiamo che l'ambiente ρ sia:

$$\begin{aligned} \rho &= [x \mapsto l_x, y \mapsto 2] \\ \Delta &= [x \mapsto \mathbf{intloc}, y \mapsto \mathbf{int}] \end{aligned}$$

e la memoria σ sia:

$$\sigma = [l_x \mapsto 3]$$

Andiamo a verificare se l'assegnamento è ben formato usando l'ambiente statico:

$$\frac{\Delta \vdash x: \text{int} \quad \Delta \vdash y: \text{int}}{\Delta \vdash x * y: \text{int}} \quad \Delta(x) = \text{intloc} \wedge \Delta(y) = \text{int}$$

$$\frac{\Delta \vdash x * y: \text{int}}{\Delta \vdash x := x * y} \quad \Delta(x) = \text{intloc}$$

cioè se $x * y$ è di tipo `int` allora $x := x * y$ è ben formata.

Esecuzione: La regola da applicare è quella per l'assegnamento:

$$\frac{\rho \vdash \langle x * y, \sigma \rangle \rightarrow_e^* \langle k, \sigma \rangle}{\rho \vdash \langle x := x * y, \sigma \rangle \rightarrow \langle x: = k, \sigma \rangle}$$

Valutiamo $x * y$, ma prima per farlo dobbiamo valutare i singoli componenti x e y :

1. Valutiamo x :

$$\frac{\rho \vdash \langle x, \sigma \rangle \rightarrow \langle l_x, \sigma \rangle}{\rho \vdash \langle x * y, \sigma \rangle \rightarrow \langle 3 * y, \sigma \rangle} \quad \rho(x) = l_x \wedge \sigma(l_x) = 3$$

2. Valutiamo y :

$$\frac{\rho \vdash \langle y, \sigma \rangle \rightarrow \langle 2, \sigma \rangle}{\rho \vdash \langle 3 * y, \sigma \rangle \rightarrow \langle 3 * 2, \sigma \rangle}$$

Ora possiamo completare la valutazione di $x * y$:

$$\rho \vdash \langle 3 * 2, \sigma \rangle \rightarrow \langle 6, \sigma \rangle$$

Si conclude applicando la regola per l'assegnamento:

$$\frac{\rho \vdash \langle x * y, \sigma \rangle \rightarrow_e^* \langle 6, \sigma \rangle}{\rho \vdash \langle x := x * y, \sigma \rangle \rightarrow \langle x: = 6, \sigma \rangle}$$

Infine applichiamo l'assioma per l'assegnamento:

$$\rho \vdash \langle x: = 6, \sigma \rangle \rightarrow_c \sigma[l_x \mapsto 6], \quad \rho(x) = l_x$$

5.6.4 Semantica dell'if

Semantica statica

La semantica statica ci determinava se il comando è ben formato e quindi va a verificare la soddisfazione di eventuali vincoli. Per quanto riguarda gli identificatori liberi e definiti:

$$\begin{aligned} FI(\text{if } b \text{ then } c_1 \text{ else } c_2) &= FI(b) \cup FI(c_1) \cup FI(c_2) \\ DI(\text{if } b \text{ then } c_1 \text{ else } c_2) &= DI(c_1) \cup DI(c_2) \end{aligned}$$

Dove nei definiti si conta anche $DI(c_1)$ a causa della presenza di blocchi contenenti dichiarazioni (da inserire nella sintassi).

$$\frac{\Delta \vdash b : \text{bool} \quad \Delta \vdash c_1 \quad \Delta \vdash c_2}{\Delta \vdash \text{if } b \text{ then } c_1 \text{ else } c_2}$$

dove l'unico vincolo di tipo è che b deve essere di tipo *bool*.

Semantica dinamica

Abbiamo le seguenti regole: Regola per l'esecuzione dell'if: per eseguire un if bisogna valutare la guardia b e poi a seconda del risultato eseguire c_1 o c_2 :

$$\frac{\rho \vdash \langle b, \sigma \rangle \rightarrow^* \langle t, \sigma \rangle}{\rho \vdash \langle \text{if } b \text{ then } c_1 \text{ else } c_2, \sigma \rangle \rightarrow \langle \text{if } t \text{ then } c_1 \text{ else } c_2, \sigma \rangle}$$

Assioma per il true: se la guardia è true allora il comando da eseguire è c_1 :

$$\rho \vdash \langle \text{if true then } c_1 \text{ else } c_2, \sigma \rangle \rightarrow \langle c_1, \sigma \rangle$$

Assioma per il false: se la guardia è false allora il comando da eseguire è c_2 :

$$\rho \vdash \langle \text{if false then } c_1 \text{ else } c_2, \sigma \rangle \rightarrow \langle c_2, \sigma \rangle$$

Esempio 5.11

rendiamo come esempio questa espressione:

$$\text{if } x = y \text{ then } x := 5 \text{ else } x := 6$$

Dove x, y sono identificatori liberi e dobbiamo fornire un ambiente e memoria per poterlo eseguire:

$$\begin{aligned}\rho &= [x \mapsto l_x, y \mapsto 2] \\ \sigma_1[l_x \mapsto 3], \sigma_2 &= [l_x \mapsto 2] \\ \Delta &= [x \mapsto \text{intloc}, y \mapsto \text{int}]\end{aligned}$$

Guardiamo la semantica statica:

$$\frac{\Delta \vdash x : \text{intloc} \quad \Delta \vdash y : \text{int} \quad \Delta \vdash x := 5 \quad \Delta \vdash x := 6}{\Delta \vdash \text{if } x = y \text{ then } x := 5 \text{ else } x := 6}$$

- Dobbiamo prima vedere se l'uguaglianza è tra interi:

$$\frac{\Delta \vdash x : \text{int} \quad \Delta \vdash y : \text{int}}{\Delta \vdash x = y : \text{bool}}$$

- Controlliamo se $x := 5$ è ben formato:

$$\frac{\Delta \vdash x : \text{int} \quad \Delta \vdash 5 : \text{int}}{\Delta \vdash x := 5}$$

- Controlliamo se $x := 6$ è ben formato:

$$\frac{\Delta \vdash x : \text{int} \quad \Delta \vdash 6 : \text{int}}{\Delta \vdash x := 6}$$

Guardiamo ora la semantica dinamica a partire da σ_1 :

$$\langle \text{if } x = y \text{ then } \dots, \sigma_1 \rangle \rightarrow ?$$

$$\frac{\rho \vdash \langle x = y, \sigma_1 \rangle \rightarrow^* \langle t, \sigma_1 \rangle}{\rho \vdash \langle \text{if } x = y \text{ then } x := 5 \text{ else } x := 6, \sigma_1 \rangle \rightarrow \langle \text{if } t \text{ then } x := 5 \text{ else } x := 6, \sigma_1 \rangle}$$

- Valutiamo x :

$$\frac{\rho \vdash \langle x, \sigma_1 \rangle \rightarrow^* \langle 3, \sigma_1 \rangle}{\rho \vdash \langle x = y, \sigma_1 \rangle \rightarrow^* \langle 3 = y, \sigma_1 \rangle}$$

- Valutiamo y :

$$\frac{\rho \vdash \langle y, \sigma_1 \rangle \rightarrow^* \langle 2, \sigma_1 \rangle}{\rho \vdash \langle 3 = y, \sigma_1 \rangle \rightarrow^* \langle 3 = 2, \sigma_1 \rangle}$$

- E quindi:

$$\rho \vdash \langle 3 = 2, \sigma_1 \rangle \rightarrow^* \langle false, \sigma_1 \rangle$$

- Riscriviamo la regola $*$ con $t = false$.

$$\frac{\rho \vdash \langle x = y, \sigma_1 \rangle \rightarrow^* \langle false, \sigma_1 \rangle}{\rho \vdash \langle \text{if } x = y \text{ then } x := 5 \text{ else } x := 6, \sigma_1 \rangle \rightarrow \langle \text{if false then } x := 5 \text{ else } x := 6, \sigma_1 \rangle}$$

- Applichiamo l'assioma per il false:

$$\rho \vdash \langle \text{if false then } x := 5 \text{ else } x := 6, \sigma_1 \rangle \rightarrow \langle x := 6, \sigma_1 \rangle$$

Eseguiamo $x := 6$:

$$\rho \vdash \langle x := 6, \sigma_1 \rangle \rightarrow \sigma_1[l_x \mapsto 6] = [l_x \mapsto 6]$$

Ora proviamo ad eseguire l'if a partire da σ_2 :

DA FARE A CASA

5.6.5 Composizione

Per comporre due comandi c_1 e c_2 usiamo il simbolo $;$, che rappresenta la composizione sequenziale. Per quanto riguarda gli identificatori liberi e definiti:

$$FI(c_1; c_2) = FI(c_1) \cup FI(c_2)$$

$$DI(c_1; c_2) = DI(c_1) \cup DI(c_2)$$

Per quanto riguarda la semantica statica, per poter comporre due comandi c_1 e c_2 è necessario che entrambi siano ben formati:

$$\frac{\Delta \vdash c_1 \quad \Delta \vdash c_2}{\Delta \vdash c_1; c_2}$$

Per quanto riguarda la semantica dinamica, per eseguire $c_1; c_2$ dobbiamo eseguire completamente c_1 e poi eseguire c_2 a partire dallo stato restituito da c_1 :

$$\frac{\rho \vdash \langle c_1, \sigma \rangle \rightarrow^* \sigma_1}{\rho \vdash \langle c_1; c_2, \sigma \rangle \rightarrow \langle c_2, \sigma_1 \rangle}$$

Eseguiamo completamente c_1 che restituisce un nuovo stato σ_1 . Da σ_1 continuiamo l'esecuzione di c_2 .

5.6.6 Comando iterativo

Il comando iterativo ci permette di ripetere sequenze di comandi. Esiste anche un altro meccanismo che garantisce la possibilità di ripetere comandi, ovvero la ricorsione, ma utilizza le procedure ed è più complesso e permette di richiamare comandi in modo non lineare, che vedremo successivamente. Esistono linguaggi che non permettono la ricorsione. Prima di parlare dell'istruzione dobbiamo parlare di:

- L'iterazione è controllata da:
 - Controllo logico: **while**
 - Controllo numerico: **for**
- Dove si trova il meccanismo di controllo (questo riguarda più il design del **while**):
 - Controllo all'inizio: **while**
 - Controllo alla fine: **do-while**

Le iterazioni si dividono in due categorie:

- Iterazione determinata, è possibile sapere quante iterazioni saranno eseguite prima di eseguirle, ad esempio con un **for** che itera su un intervallo di numeri. Questa contiene una variabile di ciclo. L'importante è sapere se la variabile può essere utilizzata all'interno del ciclo e capire la legalità della modifica della variabile di ciclo. Quando avviene la valutazione dei valori di riferimento della variabile di ciclo.

for indice = inizio *to* fine *by* c

Dove

- **indice** è la variabile di ciclo, che può essere utilizzata all'interno del ciclo, ma non può essere modificata.
 - **inizio** è il valore iniziale della variabile di ciclo, che viene valutato una sola volta all'inizio del ciclo.
 - **fine** è il valore finale della variabile di ciclo, che viene valutato una sola volta all'inizio del ciclo.
 - **by** è il passo di incremento della variabile di ciclo
- Iterazione indeterminata, non è possibile, in generale determinare a priori quanti iterazioni verranno eseguite. Non vi è una variabile di ciclo, solo un controllo dell'iterazione determinato da una guardia logica. Ci interessa di:
 - Quando avviene un controllo: se il controllo avviene all'inizio o alla fine del ciclo.
 - La forma della guardia

Semantica statica

L'unico vincolo che abbiamo è sulla guardia quindi gli identificatori sono abbastanza standard:

- Identificatori liberi:

$$FI(\text{while } e \text{ do } c) = FI(e) \cup FI(c)$$

- Identificatori definiti:

$$DI(\text{while } b \text{ do } c) = DI(c)$$

- Regola statica:

$$\frac{\Delta \vdash e : \text{bool} \quad \Delta \vdash c}{\Delta \vdash \text{while } e \text{ do } c}$$

Semantica dinamica

- Dobbiamo innanzitutto valutare la guardia:

$$\frac{\rho \vdash \langle e, \sigma \rangle \rightarrow^* \langle t, \sigma \rangle}{\rho \vdash \langle \text{while } e \text{ do } c, \sigma \rangle \rightarrow \langle \text{while } t \text{ do } c, \sigma \rangle}$$

e sarà rivalutata ad ogni iterazione. Il ciclo while con una guardia falsa si comporta come uno `skip`.

$$\rho \vdash \langle \text{while false do } c, \sigma \rangle \rightarrow \sigma$$

Il problema è quando la guardia è vera:

$$\rho \vdash \langle \text{while true do } c, \sigma \rangle \rightarrow \langle c; \text{while } e \text{ do } c, \sigma \rangle$$

Riscriviamo le regole in una forma più corretta, fondiamo la regola di valutazione con gli assiomi.

$$\frac{\rho \vdash \langle e, \sigma \rangle \rightarrow^* \langle \text{false}, \sigma \rangle}{\rho \vdash \langle \text{while } e \text{ do } c, \sigma \rangle \rightarrow \sigma}$$
$$\frac{\rho \vdash \langle e, \sigma \rangle \rightarrow^* \langle \text{true}, \sigma \rangle}{\rho \vdash \langle \text{while } e \text{ do } c, \sigma \rangle \rightarrow \langle c; \text{while } e \text{ do } c, \sigma \rangle}$$

Esempio 5.12 (P)

endiamo come esempio questo comando:

$$\text{while not } x = 0 \text{ do } x := x - 1$$

dove x è un identificatore libero e dobbiamo fornire un ambiente e memoria per poterlo

eseguire:

$$\rho = [x \mapsto l_x]$$

$$\Delta = [x \mapsto \text{intloc}]$$

$$\sigma = [l_x \mapsto 2]$$

Per la semantica statica: DA FARE A CASA Per la semantica dinamica:

5.7 Blocco

Nel nostro linguaggio introduciamo il costrutto del blocco per raggruppare comandi e dichiarazioni, potenzialmente isolata da altri pezzi di codice.

- L'idea è quella di rendere modulare il programma e quindi più chiaro da comprendere.
- Permette inoltre la gestione locale di identificatori.
- Permette di gestire l'allocazione della memoria.

5.7.1 Blocco inline

Il blocco inline (senza nome) ci permette di specificare le dichiarazioni D che definiscono l'ambiente in cui eseguire la porzione C .

$$D ; C$$

In alcuni linguaggi di programmazione per evidenziare il blocco viene delimitato da parentesi graffe.

$$\{ \text{blocco} \}$$

In **IMP** invece abbiamo i comandi che possiedono semanticamente la composizione sequenziale, quindi è possibile inserire dichiarazioni in qualunque punto. Con questa sintassi (e con la semantica che specificheremo) una dichiarazione è visibile da dove viene inserita fino alla fine del programma. I blocchi possono essere annidati.

💡 Osservazione 5.1

Se apro un blocco dentro ad un altro blocco, non posso chiuderlo prima di aver chiuso il blocco esterno. Devo sempre chiudere l'ultimo blocco aperto. I blocchi possono essere solo *completamente annidati*.

Questa regola di sovrapposizione dei blocchi permette di definire la regola di visibilità delle dichiarazioni.

5.7.2 Regola di visibilità

Una dichiarazione locale ad un blocco è visibile nel blocco e in tutti i blocchi annidati, a meno che in tali blocchi non intervenga una dichiarazione dello stesso nome.

```
1 {  
2   const i : int = 3; // dichiarazione globale  
3   {  
4     var i : real = 1.2;  
5     i = i + 1;  
6   }  
7   i = i + 1;  
8 }
```

In questo esempio, la dichiarazione di *i* all'interno del blocco annidato nasconde la dichiarazione globale di *i*. Quindi, all'interno del blocco annidato, *i* si riferisce alla variabile di tipo **real**, mentre all'esterno del blocco annidato, *i* si riferisce alla costante di tipo **int**. La porzione di codice annidata si chiama **scope di i**.

Esempio 5.13

Prendiamo come esempio questo codice:

```
1 A: {  
2   int a = 1;  
3   B: {  
4     int b = 2;  
5     int c = 2;  
6     C : {  
7       int c = 3;  
8       int d;  
9       d = a + b + c;  
10      write(d);  
11    }  
12    D : {  
13      int e;  
14      e = a + b + c;  
15      write(e);  
16    }  
17  }  
18 }
```

In questo caso:

- il blocco *A* è globale, visibile in tutti i blocchi essendo tutti i blocchi annidati in *A*.
- Locali al blocco *B* sono visibili in *B*, *C* e *D*.
- Locali al blocco *C* sono visibili solo in *C*. Nasconde dentro *C* la precedente dichiarazione di *c* in *B*.

- Locali al blocco D sono visibili solo in D e non vede C .

✚ Definizione 5.17: Scope di una dichiarazione di x

Porzione di codice in cui tutte le occorrenze di x fanno riferimento alla dichiarazione. Devono usare il legame definito dalla dichiarazione. Lo **scope** è il concetto statico che lega un identificatore all'ambiente di riferimento a tempo di compilazione in quanto (senza l'uso di procedure) **eseguimo** il codice dove viene **definito**.

Di conseguenza una **variabile** può essere:

- Globale quando è visibile da tutti i blocchi, quindi l'intero programma
- Rispetto al blocco in cui si trova le variabili possono essere:
 - Locali, ovvero inserite dentro il blocco, che sovrascrivono eventuali esterne
 - Non locali (globali incluse) visibili al blocco ma posizionate in blocchi esterni.

Gli **ambienti** possono essere:

- Globali, ambiente visibile a tutti i blocchi del programma
- Locali, l'ambiente è definito nel blocco e non visibile esternamente
- Non locali, l'ambiente ereditato dai blocchi esterni

✚ Definizione 5.18: Scoping

Lo scoping è un concetto che riguarda porzioni di codice, determina a quali legami (ambiente statico) fa riferimento ogni occorrenza.

5.7.3 Tempo di vita

Ogni variabile ha un tempo di vita ovvero il tempo di esecuzione in cui l'associazione della locazione (memoria) rimane valida. Il tempo di vita è un **concetto dinamico** perché dipende dall'esecuzione del programma. Il tempo di vita è delimitato da:

- Allocazione in memoria
- Deallocazione

Se l'allocazione e deallocazione non è esplicita, allora il tempo di vita coincide con il tempo di esecuzione del programma.

Esempio 5.14

Prendiamo come esempio questo codice:

```
1 {  
2   const i : int = 3; // dichiarazione globale  
3   {  
4     var i : real = 1.2;  
5     i = i + 1;  
6   }  
7   i = i + 1;  
8 }
```

- Il tempo di vita di **const** *i* è l'intero programma
- Il tempo di vita di **var** *i* è limitato al blocco annidato,

Esempio 5.15

Prendiamo come esempio questo codice:

```
1 {  
2   while ...  
3   {  
4     D1 dich.  
5     C11 comandi  
6     {  
7       D2 dich.  
8       C12 comandi  
9     }  
10    C13 comandi  
11  }  
12 }
```

Si adatta esattamente al concetto di stack di attivazione, dove ogni blocco annidato rappresenta un nuovo frame nello stack.

1. Passando dal primo blocco, lo stack viene allocato in questa maniera:

$$\rightarrow [\rho]$$

2. Passando al blocco D1, lo stack viene allocato in questa maniera:

$$\rightarrow [\rho_1, \rho]$$

3. Entrando dentro al blocco D2:

$$\rightarrow [\rho_2, \rho_1, \rho]$$

Il tempo di vita delle dichiarazioni D_2 è la durata dell'ambiente ρ_2 nello stack.

4. Uscendo dal blocco D2, lo stack torna a questa configurazione:

$$\rightarrow [\rho_1, \rho]$$

E abbiamo deallocato ρ_2 e poi così anche per ρ_1 . Allocazioni di ρ_1 e ρ_2 successive nello stack durante l'esecuzione del while sono di fatto ambienti nuovi, diversi.

5.7.4 Semantica statica

Ricordiamo che la semantica statica controlla se il comando è ben formato.

$$\frac{\Delta \vdash D : \Delta_0 \quad \Delta[\Delta_0] \vdash C}{\Delta \vdash D; C}$$

Per quanto riguarda gli identificatori liberi e definiti:

$$\begin{aligned} FI(D; C) &= FI(D) \cup (FI(C) \setminus DI(D)) \\ DI(D; C) &= DI(D) \cup DI(C) \end{aligned}$$

5.7.5 Semantica dinamica

Regola per l'esecuzione delle dichiarazioni di un blocco:

$$\frac{\rho \vdash \langle D, \sigma \rangle \rightarrow^* \langle \rho_0, \sigma' \rangle}{\rho \vdash \langle D; C, \sigma \rangle \rightarrow^* \sigma'}$$

Regola per l'esecuzione del blocco:

$$\frac{\rho[\rho_0] \vdash \langle C, \sigma \rangle \rightarrow^* \sigma'}{\rho \vdash \langle D; C, \sigma \rangle \rightarrow^* \sigma'}$$

Esempio 5.16

Consideriamo il seguente codice:

$$\underbrace{\text{var } x : \text{int} = 3;}_d \underbrace{x := x + 1}_c$$

Proviamo a vedere se il comando è ben formato. **Semantica statica:**

$$\frac{\vdash d : \Delta \quad \Delta \vdash c}{\vdash d; c}$$

Espandiamo per $\vdash d : \Delta$:

$$\frac{\vdash 3 : int}{\vdash \text{var } x : int = 3 : [x \mapsto intloc] \equiv \Delta}$$

Riscriviamo la regola per l'esecuzione del blocco:

$$\frac{\vdash d : [x \mapsto intloc] \quad [x \mapsto intloc] \vdash c}{\vdash d; c}$$

Espandiamo per $[x \mapsto intloc] \vdash c$:

$$\frac{[x \mapsto intloc] \vdash x + 1 : ?}{[x \mapsto intloc] \vdash x := x + 1}$$

Espandiamo per $[x \mapsto intloc] \vdash x + 1 : ?$:

$$\frac{[x \mapsto intloc] \vdash x : int \quad [x \mapsto intloc] \vdash 1 : int}{[x \mapsto intloc] \vdash x + 1 : int}$$

Ora proviamo a eseguire il blocco. **Semantica dinamica:**

$$\frac{\dots}{\vdash \langle d; c, \emptyset \rangle}$$

5.8 Procedure

L'obiettivo è quello di **astrarre il controllo**, cioè nascondere dei dettagli implementativi fornendo al programmatore un'interfaccia più semplice da utilizzare. Per fare ciò bisogna **identificare cosa è necessario descrivere** e mantenere pubblico e cosa si può nascondere, in modo da poter focalizzare la programmazione sulle questioni rilevanti. Questa astrazione permette di:

- Ottimizzare l'implementazione
- Ridurre gli errori
- Migliorare la leggibilità

L'astrazione del controllo **introduce dei legami**, cioè dei **blocchi di codice con un nome**, che vanno gestiti separando la definizione dall'esecuzione.

Gli aspetti principali dell'astrazione del controllo sono:

- **Nome:** si riferisce ad un **insieme di comandi**, ma può essere usato in vari modi. Essenzialmente il nome identifica un blocco di codice che **esegue sempre nello stesso modo**
- **Parametri:** permettono di creare un "ponte" tra l'ambiente che **chiama** e l'ambiente **eseguito** che permette di passare informazioni tra i due ambienti. Essenzialmente permettono di usare un pezzo di codice su valori (input) differenti ed eseguire computazioni simili usando lo stesso nome.

Il nome, i parametri e il **tipo di risultato** rappresentano l'**interfaccia** di un blocco di codice, che è la parte pubblica esposta al programmatore. La parte nascosta dell'astrazione è il **corpo della procedura**, cioè il codice eseguito.

Esempio 5.17

Un esempio di procedura, un tipo di astrazione del controllo, è la seguente:

```
      {
double P(int x) {
    double z;
    /* corpo */
    return z;
}
      }
```

Corpo

Per definire procedure il linguaggio di programmazione deve avere strumenti sintattici per:

- Specificare (definire) la procedura (interfaccia)
- Scrivere il corpo della procedura (corpo), che insieme all'interfaccia rappresenta la **definizione completa di una procedura**
- Utilizzare la procedura, cioè chiamare l'esecuzione del corpo della procedura

La definizione viene fatta tramite **dichiarazioni**, mentre l'utilizzo viene fatto tramite **comandi**.

Esempio 5.18

Consideriamo il seguente codice:

```
      Specifica
      int foo(int n, int a) {
          int tmp = a;
          if (tmp == 0) then return n;
          else return n + 1;
      }
      :
      { // contesto del chiamante

          int x = foo ( 3, 0 );
                    Nome Parametri
                    attuali

      }
```

dichiarazione

comando

- Il **blocco** definisce un ambiente locale che contiene:
 - Dichiarazioni interne (ad esempio `tmp`)
 - Parametri formali (`n` e `a`)
- I parametri attuali sono valori con cui si vogliono inizializzare i parametri formali della procedura
- Il tipo della procedura e il tipo del risultato devono coincidere sia nella definizione che nell'utilizzo

5.8.1 Ambiente di riferimento di una procedura

L'ambiente di riferimento di una procedura è determinato da:

- **Regole di visibilità:** standard per l'annidamento
- **Regole di scoping:** specificano dove risolvere ambienti non locali di codice eseguito su contesti diversi da quello di definizione
 - **Regole di binding:** risolve regole di scoping ambigue, ad esempio nel caso di una funzione come parametro ad un'altra funzione
- **Regole di passaggio di parametri**
- **Regole specifiche del linguaggio**

5.8.2 Regole di scoping

Consideriamo il seguente codice:

```
int foo(int n){  
    int tmp = a;  
    if (tmp == 0) then return n;  
    else return n + 1  
}
```

In questo caso la variabile **a** è un **identificatore non locale**, cioè un identificatore libero per cui si deve trovare il significato nel contesto. Il problema è che nel contesto del chiamante potrebbero esserci più definizioni di **a** e quindi bisogna gestire quale utilizzare. Di conseguenza la regola di visibilità delle dichiarazioni ?? non è più sufficiente a risolvere i riferimenti quando si usano le procedure se la procedura ha un ambiente non locale.

Esempio 5.19

Nel seguente codice la variabile **x** è dichiarata correttamente, ma non sappiamo a quale dichiarazione la funzione fa riferimento:

```
1 {  
2     int x = 3;  
3     int foo() {  
4         return x;  
5     }  
6  
7     {  
8         int x = 5;  
9         int res = foo(); // a quale x fa riferimento foo()?  
10    }  
11 }
```

La risoluzione di questo problema è gestita diversamente da ogni linguaggio di programmazione. Per risolverlo va definita la **regola di scoping** che può essere:

- **Statico**: l'ambiente è quello della definizione
- **Dinamico**: l'ambiente è quello della chiamata

Le regole di scoping quindi servono per risolvere riferimenti **non locali** presenti in procedure, ovvero dove il contesto di definizione può non coincidere con il contesto di esecuzione.

5.8.3 Scoping statico

Esempio 5.20

Consideriamo il seguente codice:

```
1 {  
2   int x = 10; // x_1  
3   void foo() {  
4     // x non e' un parametro formale  
5     // e non e' dichiarata nel blocco  
6     x++; // Ambiente non locale  
7   }  
8   void fie() {  
9     int x = 0; // x_2  
10    foo();  
11  }  
12  
13  fie();  
14 }
```

Non possiamo sapere se alla fine si ottiene $x = 1$ (scoping dinamico) o $x = 11$ (scoping statico).

- Scoping statico: si sceglie l'ambiente di definizione a tempo di compilazione, quindi utilizza la dichiarazione di x nello scope della definizione della procedura
- Scoping dinamico: si sceglie l'ambiente di chiamata a tempo di esecuzione, quindi utilizza la dichiarazione di x nello scope della chiamata

Consideriamo il seguente codice:

```
1 def(P) {  
2   def(x)  
3  
4   def(P1) {  
5     def(P11) {  
6       use(x)  
7     }  
8  
9     P11()  
10  }  
11  
12  def(P2) {  
13    def(x)  
14  }
```

```

15   P1()
16   }
17
18   P2()
19 }

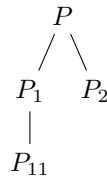
```

Nella sequenza di **annidamento statico** (delle definizioni) P_{11} non vede P_2 , quindi l'uso di x in P_{11} si riferisce alla definizione di x in P_1 .

$$P : \begin{bmatrix} P_1 : \begin{bmatrix} P_{11} : [\end{bmatrix} \\ P_2 : [\end{bmatrix}$$

Lo scoping statico consiste nell'applicare la regola di visibilità usando come relazione di annidamento **quello di definizione**.

- Sequenza di definizione:



La visibilità di P_{11} è limitata a P_1 . Grazie a questo la risoluzione dei riferimenti avviene a tempo di compilazione

- Sequenza di chiamate:

$$P \rightarrow P_2 \rightarrow P_1 \rightarrow P_{11}$$

Esempio 5.21

Consideriamo il seguente esempio:

```

1 {
2   int x = 0;
3   void pippo(int n) {
4     x = n+1; // x non locale, n locale
5   }
6
7   pippo(3); // x = 4
8   write(x); // 4
9   {
10    int x = 0; // <-+

```

```

11     pippo(4); // -|---> x = 5
12     write(x); // --+ 0
13 }
14 write(x); // 5
15 }

```

1. Alla prima esecuzione di `pippo(3)` si modifica `x` globale e si ottiene `x = 4`
2. Alla seconda esecuzione di `pippo(4)` si modifica ancora `x` globale e il valore stampato è quello della `x` locale, quindi 0
3. L'ultima `write` stampa il valore di `x` globale, quindi 5

🔗 Definizione 5.19: Scoping statico

Lo scoping statico garantisce l'indipendenza del riferimento dalla posizione della chiamata.

- Rende il codice più comprensibile
- Ma è più difficile da implementare

🔗 Esempio 5.22

Alcuni esempi di scoping statico in linguaggi di programmazione comuni sono i seguenti:

1. JavaScript:

- Esempio 1

```

1  var n = 12;
2
3  function addn() {
4      return n + 30;
5  }
6
7  console.log(n + '\n');
8
9  n = addn();
10
11 console.log(n);

```

In output si ottiene:

```

1  12
2
3  47

```

- Esempio 2

```
1 var n = 12;
2
3 function addn() {
4     return n + 30;
5 }
6
7 console.log(n + '\n');
8
9 var n = 17;
10 n = addn();
11
12 console.log(n);
```

In output si ottiene:

```
1 12
2
3 47
```

In entrambi i casi viene sovrascritta la variabile **n** globale

- Esempio 3

```
1 var n = 12;
2
3 function addn() {
4     return n + 30;
5 }
6
7 function setn() {
8     var n = 17;
9     n = addn();
10    console.log(n + '\n');
11 }
12
13 setn();
14
15 console.log(n);
```

In output si ottiene:

```
1 42
2
3 12
```

Da questo esempio si nota che JavaScript utilizza scoping statico

2. Python:

- Esempio 1

```
1 n = 12;
2
3 def addn():
4     return(n + 30);
5
6 def setn():
7     n = 17;
8     n = addn();
9     print(n);
10
11 setn();
12 print(n);
```

In output si ottiene:

```
1 42
2
3 12
```

- Esempio 2

```
1 n = 12;
2
3 def addn():
4     return(n + 30);
5
6 def setn():
7     n = 17;
8     n = addn();
9     print(n);
10
11 n = 13;
12 setn();
13 print(n);
```

In output si ottiene:

```
1 43
2
3 13
```

- Esempio 3

```
1 x = 4;
2 z = 3;
3
4 def f(y):
5     return(x * y);
6
```

```

7 def setn():
8     x = 5;
9     print(f(z) + x)
10
11 setn();
12 print(x);

```

In output si ottiene:

```

1 17
2 4

```

Anche python utilizza scoping statico

Lo scoping statico è più difficile da implementare, ma è più efficiente e garantisce una maggiore comprensibilità del codice, inoltre è calcolabile a tempo di compilazione.

5.8.4 Scoping dinamico

Consideriamo il seguente codice:

```

1 def(P) {
2     def(x)
3
4     def(P1) {
5         def(P11) {
6             use(x)
7         }
8
9         P11()
10    }
11
12    def(P2) {
13        def(x)
14
15        P1()
16    }
17
18    P2()
19 }

```

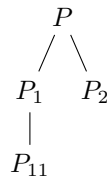
Lo scoping dinamico consiste nell'applicare la regola di visibilità usando come relazione di anidamento quella della **sequenza di chiamate**.

- Sequenza di chiamate:

$$P \rightarrow P_2 \rightarrow P_1 \rightarrow P_{11}$$

P_{11} cerca la definizione di x in P_1 e non la trova, quindi cerca in P_2 e la trova, quindi utilizza quella definizione di x a tempo di esecuzione

- Sequenza di definizione:



Il riferimento (legame) corretto **dipende dalla posizione della chiamata**, quindi chiamate diverse possono fare riferimento a legami diversi.

Esempio 5.23

Consideriamo il seguente codice:

```

1 {
2   int x = 0;
3   void pippo(int n) {
4     x = n + 1;
5   }
6
7   pippo(3);
8   write(x); // 4
9
10  {
11    int x = 0; // <--+<+
12    pippo(4); // --+ |
13    write(x); // 5 --+
14  }
15  write(x); // 4
16 }
```

La dichiarazione di x utilizzata da **pippo** dipende da dove viene chiamata, quindi:

- Alla prima chiamata di **pippo(3)** viene modificata la x globale e si ottiene $x = 4$
- Alla seconda chiamata di **pippo(4)** viene modificata la x locale e si ottiene $x = 5$
- L'ultima write stampa il valore di x globale, quindi 4

Le cause che rendono due risultati diversi in base allo scoping sono:

- Il codice contiene una procedura con un ambiente non locale
- Esistono diverse definizioni di questi riferimenti non locali nel codice

Esempio 5.24

Alcuni esempi di scoping dinamico in linguaggi di programmazione comuni sono i seguenti:

1. Lisp:

- Esempio 1

```

1 (defvar n 12)
2
3 (defun addn () (+ n 30))
4
5 (defun setn ()
6   (let ((n 17))
7     (print (addn))))
8 )
9
10 (setn)
11 (print n)
```

In output si ottiene:

```

1 47
2 12
```

- Esempio 2

```

1 (defvar x 4)
2 (defvar z 3)
3
4 (defun f (y) (* x y))
5
6 (let ((x 5))
7   (print (+ (f z) x)))
8
9 (print x)
```

In output si ottiene:

```

1 20
2 4
```

2. Perl:

- Esempio 1

```

1 $n = 12;
2
3 sub addn() {return $n + 30};
4
5 sub setn() {
6     $n = 17;
7     $n = addn();
8     print $n
9 };
10
11 setn();
12
13 print ' ';
14 print $n;

```

In output si ottiene:

```

1 47 47

```

In questo codice non c'è alcun tipo di scoping perchè le variabili sono tutte globali. Per avere variabili locali bisogna utilizzare la parola chiave `local`.

- Esempio 2

```

1 $n = 12;
2
3 sub addn() {return $n + 30};
4
5 sub setn() {
6     local $n = 17;
7     $n = addn();
8     print $n
9 };
10
11 setn();
12
13 print ' ';
14 print $n;

```

In output si ottiene:

```

1 47 12

```

Se si forza la località delle variabili si ottiene scoping dinamico.

Lo scoping dinamico ha un'implementazione più semplice, ma rende il codice meno leggibile e siccome è eseguito a tempo di esecuzione è più difficile fare analisi statica del codice.

Semantica statica: Verifichiamo che l'if sia ben formato usando l'ambiente statico:
Per prima cosa verifichiamo che le espressioni e le dichiarazioni siano ben formate:

1. Guardia

$$\frac{\Delta \vdash x : \text{int} \quad \Delta \vdash y : \text{int}}{\Delta \vdash x = y : \text{bool}} \quad \Delta(x) = \text{intloc} \wedge \Delta(y) = \text{int}$$

2. Ramo then

$$\frac{\Delta \vdash x : \text{int}}{\Delta \vdash x := 5} \quad \Delta(x) = \text{intloc}$$

3. Ramo else

$$\frac{\Delta \vdash x : \text{int}}{\Delta \vdash x := 6} \quad \Delta(x) = \text{intloc}$$

Dopo aver verificato tutte le componenti dell'if possiamo dire che è ben formato:

$$\frac{\Delta \vdash x = y : \text{bool} \quad \Delta \vdash x := 5 : \text{int} \quad \Delta \vdash x := 6 : \text{int}}{\Delta \vdash \text{if } x = y \text{ then } x := 5 \text{ else } x := 6}$$

5.9 Allocazione della memoria

Un altro aspetto importante nel creare un linguaggio di programmazione è come e quando viene allocato lo spazio in memoria per gli elementi del programma. Ciò ha effetto sugli algoritmi di risoluzione dei riferimenti non locali. Si distinguono due tipi di allocazione:

- **Allocazione statica:** La memoria viene allocata a tempo di compilazione
- **Allocazione dinamica:** La memoria viene allocata a tempo di esecuzione.

Vi sono due strutture in cui questi dati vengono allocati:

- **Stack**
- **Heap**

Esempio 5.25

In FORTRAN questo programma è sintatticamente illegale: ricorsione non ammessa:

```
1 SUBROUTINE ERROR(N)
2   IF (N .LE. 0) THEN
3     CALL ERROR(N-1)
4   PRINT N
5   END
6
7 xxx CALL ERROR(3)
```

Questo succede perchè in FORTRAN l'allocazione è statica, quindi non è possibile allocare memoria per la chiamata ricorsiva di `ERROR` a tempo di esecuzione.

5.9.1 Allocazione dinamica su pila

La pila è una struttura dati allocata dinamica e gestita in modalità FIFO. Per ogni chiamata a procedura viene creato un RdA (record di attivazione) il quale contiene tutte le informazioni per eseguire la nostra procedura e reimpostare lo stato della macchina nel momento in cui la procedura è terminata. L'RdA contiene:

- Indirizzo di ritorno
- Variabili locali (parametri formali e ambienti dichiarato nella procedura)
- Puntatore alla testa della pila
- Altre informazioni specifiche:
 - Il puntatore statico è un puntatore al RdA della procedura genitore nella catena storica delle definizioni. Può essere distante nello stack dell'RdA corrente.
 - Il puntatore dinamico è il puntatore dell'RdA della procedura chiamante.

5.9.2 Procedura di chiamata a funzione

Quando viene chiamata una funzione bisogna:

- Valutare i parametri (in funzione del metodo di passaggio)
- Allocare RdA sullo stack con ambiente locale
- Salva lo stato del chiamante al momento della chiamata
- Trasferimento del controllo alla procedura chiamata

Procedura di ritorno da funzione

Quando la procedura chiamata termina bisogna:

- Recupero informazioni dal RdA (passaggio valore-risultato)
- Deallocazione RdA dallo stack
- Ripristino dello stato del chiamante al momento della chiamata
- Ritorno del controllo al chiamante

5.9.3 Implementazione dello scoping statico

Per implementare lo scoping statico dobbiamo introdurre il concetto di **link statico** che punta all'indirizzo del RdA della procedura che contiene la definizione della procedura in atto.

📌 Definizione 5.20: Profondità statica

La profondità statica è il valore che denota la profondità di annidamento.

🔧 Esempio 5.26

Immaginiamo le seguenti procedure A, B, C, D, E .

$A \xrightarrow{\text{link dinamico}} B \rightarrow C \rightarrow D \rightarrow E \rightarrow C$ (sequenze di chiamate)

```
1 A() {  
2   B() {  
3     C();  
4   }  
5   C() {  
6     D() {  
7       E();  
8     }  
9     E() {  
10      C();  
11    }  
12    D();  
13  }  
14 }
```

- B e C sono definiti dentro A
- D e E è definito dentro C

Quindi la profondità statica di A è 0:

$$sd(A) = 0 \quad sd(B) = 1 + sd(A) = 1 = sd(C) \quad sd(D) = 1 + sd(C) = 2 = sd(E)$$

Quali informazioni statiche abbiamo per calcolare il link statico:

- Profondità di annidamento

- RdA della procedura (struttura nota sintatticamente)

Come calcolare il link statico (Ch denota il chiamante e CS denota la catena statica):
Dobbiamo capire chi chiama P : dobbiamo determinare link statico di P :

- Se P è definito in Ch (link statico = link dinamico)
- Se P è definito in un blocco che si trova k passi prima di Ch dobbiamo risalire la catena statica di k passi.

$$k = sd(Ch) - sd(P) + 1$$

- Se $k = 0$ allora Ch def. P e quindi $CS(P = Ch)$ Il link statico per P è l'indirizzo del chiamante.
- Se $k > 0$ allora $CS(P) = \overbrace{CS(CS \dots CS(Ch))}^{k \text{ volte}}$. Il link statico si ottiene risalendo k volte la catena statica.

Quindi proviamo ad applicarlo nel nostro esempio e calcoliamo i link statici per ogni procedura:

$$k_B = sd(A) - sd(B) + 1 = 0 - 1 + 1 = 0 \Rightarrow CS(B) = A$$

$$k_C = sd(B) - sd(C) + 1 = 1 - 1 + 1 = 1 \Rightarrow CS(C) = CS(B)$$

$$k_D = sd(C) - sd(D) + 1 = 1 - 2 + 1 = 0 \Rightarrow CS(D) = C$$

$$k_E = sd(D) - sd(E) + 1 = 2 - 2 + 1 = 1 \Rightarrow CS(E) = CS(D)$$

Calcolo link statico di C chiamato da E :

$$k_C = sd(E) - sd(C) + 1 = 2 - 1 + 1 = 2$$

$$\Rightarrow CS(C) = CS(CS(E)) = CS(CS(D)) = CS(B) = A$$

5.10 Risoluzione dei riferimenti non locali

Nota: se il riferimento è locale, la risoluzione è ovvia nell'RdA attivo. Per esempio, se abbiamo x usata in P ma non locale dobbiamo individuare il blocco più vicino nella catena statica che definisce x :

- Usiamo sd e l'informazione data dai blocchi che definiscono x .
- Usiamo la catena statica

Denoteremo con N_x quanto dobbiamo risalire la catena statica per trovare la corretta x .

$$N_x = sd(A) - sd(main) = 1 - 0 = 1$$

Dove:

- A è il blocco che usa x
- $main$ è il blocco più vicino ad A che definisce x nella catena di annidamento

5.11 Implementazione scoping dinamico

Ci sono diversi algoritmi per implementarlo ma noi useremo il **Shallow Access** che utilizza strutture dati centrali basate su CRT (Tabella centrale dei riferimenti o Central Reference Table):

- Permette di evitare ricerche lineari nella catena dinamica
- Mantiene in modo centralizzato le informazioni necessarie per accedere all'RdA dinamico corretto.
- È una tabella con una voce per ogni identificatore usato nel programma
- Ogni identificatore punta ad ogni lista tale che in testa si trovano le informazioni dell'ultimo RdA che definisce l'identificatore (dinamico)

Esempio 5.27

Consideriamo il seguente ordine di chiamate:

$$A \rightarrow B \rightarrow C \rightarrow D$$

Costruiamo lo schema statico di annidamento delle definizioni (Scoping statico)

```
1 A() {  
2   x, y  
3   B() {  
4     x, y  
5   }  
6   C() {  
7     w, x  
8     D() {  
9       w  
10    }  
11  }  
12 }
```

- Chiamata di A :
 - Creazione dell'RdA per A sullo stack

- Abbiamo indirizzo di RdA di A (e offset per x)
- Chiamata di B :
 - Creazione dell'RDA per B sullo stack
 - Aggiornamento CRT: x e y puntano all'RDA di B
- Chiamata di C :
 - Creazione dell'RDA per C sullo stack
 - Aggiornamento CRT: w e x puntano all'RDA di C
- Chiamata di D :
 - Creazione dell'RDA per D sullo stack
 - Aggiornamento CRT: w punta all'RDA di D

All'uscita da un blocco viene distrutto l'RdA e viene eliminata la testa di ogni lista linkata da identificatori definiti nel blocco.

5.12 Procedure in IMP

In un linguaggio di programmazione le procedure:

- vanno **definite** (modifica delle dichiarazioni)
- vanno **chiamate** (modifica dei comandi)

Dichiarazioni

Questo è il modo in cui dichiareremo le procedure in IMP:

$$D \rightarrow \dots \mid \text{proc } P () C$$

dove:

- **proc** è una parola chiave che indica la dichiarazione di una procedura
- P è un identificatore di procedura
- C è un comando che rappresenta il corpo della procedura
- In questo caso non ci sono parametri formali, ma è possibile aggiungerli

Aggiungiamo agli oggetti denotabili le "astrazioni" cioè funzioni:

$$\lambda \varepsilon. C$$

Quindi utilizziamo la notazione del lambda calcolo per indicare una funzione e ε vuol dire non ci sono parametri.

5.12.1 Regole per le dichiarazioni

$$\rho \vdash \langle \text{proc } P() \ C, \sigma \rangle \rightarrow \left\langle \overbrace{[\rho \mapsto \lambda\varepsilon.C']}^{\text{associazione costante di } P \text{ al suo codice}}, \sigma \right\rangle$$

$$C' = \begin{cases} \rho; C & \text{se scoping statico} \\ C & \text{se scoping dinamico} \end{cases}$$

Quindi:

- Se è scoping **statico**, fisso l'ambiente presente al momento della definizione di P per averlo alla sua esecuzione.
- Se è scoping **dinamico**, si usa l'ambiente alla chiamata P

Comandi

Questo è il modo in cui invocheremo le procedure in IMP:

$$C \mapsto \dots \mid P()$$

dove:

- P è un identificatore di procedura
- $()$ indica che stiamo chiamando la procedura P senza parametri

5.12.2 Regole per la chiamata

$$\bar{\rho} \vdash \langle P(), \sigma \rangle \rightarrow \langle C', \sigma \rangle$$

con

$$\bar{\rho}(P) = \lambda\varepsilon.C'$$

dove C' può essere in base allo scoping:

- Se statico $C' = \rho; C$
- Se dinamico $C' = C$

Con definizione `proc P() C` nell'ambiente ρ .

Esempio 5.28 (Esercizio)

Consideriamo il seguente codice in IMP:

```
1 {  
2   int b;  
3   void A() {  
4     int x = 0, y = 0, z = 5;  
5     void B () {  
6       int x = 3, w = 3;  
7       x = y + z;  
8     }  
9     void C (int a) {  
10      int y = 1, v = 1;  
11      void D() {  
12        int z = 2, v = 2;  
13        B();  
14        v = x + y;  
15      }  
16      D();  
17      x = z + y;  
18    }  
19    C(7);  
20  }  
21  A();  
22 }  
23 Esercizio sul quaderno.
```

5.13 Passaggio dei parametri

Le procedure senza parametri sono procedure costanti possono essere viste come zucchero sintattico per parti di codice che vengono usate molteplici volte, al contrario funzioni con parametro possono dare diverso significato alle stesse procedure, quindi è necessario introdurre il concetto di passaggio dei parametri. I **parametri** sono come "input" delle procedure che permettono una comunicazione tra il chiamante e il chiamato.

Dichiarazione di una funzione con parametri:

$$\underbrace{\text{void}}_{\text{valore ritorno}} \quad \text{fun}(\underbrace{\text{int } x, \text{int } y}_{\text{parametri formali}})$$

Chiamata di una funzione con parametri:

$$\text{fun} \quad \underbrace{(5, 10)}_{\text{parametri attuali}} \quad \text{oppure} \quad \text{fun} \quad \underbrace{(x, y)}_{\text{parametri attuali}}$$

Il tipo di comunicazione all'interno dei parametri può essere diverso a seconda del **metodo di passaggio dei parametri utilizzato**, i principali sono:

- **Passaggio per valore:** Il chiamante passa una copia del valore del parametro al chiamato.
- **Passaggio per riferimento:** Il chiamante passa un riferimento (indirizzo) al parametro al chiamato, permettendo al chiamato di modificare il valore del parametro nel chiamante.
- **Passaggio per risultato:** Il chiamante passa una variabile non inizializzata al chiamato, che la utilizza per restituire un valore al chiamante.
- **Passaggio per valore-risultato:** Combina il passaggio per valore e per risultato.

Un altro aspetto importante è l'**associazione tra parametri formali e parametri attuali**, che può variare:

5.13.1 Modalità di passaggio di parametri: direzioni

Le direzioni dei parametri indicano se un parametro è usato per fornire dati al chiamato, restituire dati al chiamante o entrambi. Le principali direzioni dei parametri sono:

- **In-mode:** Il parametro è usato solo per fornire dati al chiamato (input).
- **Out-mode:** Il parametro è usato solo per restituire dati al chiamante (output).
- **InOut-mode:** Il parametro è usato sia per fornire dati al chiamato che per restituire dati al chiamante.

E le modalità di passaggio dei parametri possono essere associate a queste direzioni in diversi modi:

- Il passaggio per valore è tipicamente associato a parametri in-mode, poiché il chiamato riceve solo una copia del valore e non può modificare il valore originale nel chiamante.
- Il passaggio per risultato è tipicamente associato a parametri out-mode, poiché il chiamato restituisce un valore al chiamante.
- Il passaggio per riferimento e per valore-risultato possono essere associati a parametri inout-mode, poiché il chiamato può sia ricevere dati dal chiamante che restituire dati al chiamante.

5.13.2 Passaggio per valore

Definizione 5.21

Il valore del parametro attuale (eventualmente tramite una valutazione di un'espressione) viene passato come valore iniziale del corrispondente parametro formale. Viene implementato **creando una copia** dell'attuale nel formale.

$$\text{fun}(\underbrace{x + 1}_{\text{il parametro attuale è un r-value}}, y + 2)$$

- Il passaggio per valore è vantaggioso perché versatile nella scrittura nei parametri attuali
- Tuttavia, se dovessimo copiare oggetti di grandi dimensioni, il passaggio per valore potrebbe essere inefficiente in termini di tempo e memoria.

A livello di implementazione si potrebbe fare il passaggio per valore utilizzando dei riferimenti che però devono essere protetti per evitare modifiche al valore originale.

Esempio 5.29

Consideriamo il seguente codice:

```
1 void foo(int x) { x = x + 1; }
2 ...
3 y = 1;
4 foo(y+1); // Chiamata a foo alloca x nell'ambiente locale di foo
```

Alla chiamata $y + 1$ viene valutato a 2 e il valore viene caricato nella cella riferita da x . Non c'è nessun legame tra y e x . Alla terminazione della procedura foo , il RdA contenente x viene distrutto perdendo tutte le modifiche fatte a x . Questa metodologia non permette di comunicare al chiamante attraverso i parametri.

Esempio 5.30

Consideriamo il seguente codice; facciamo finta che si possa specificare il tipo di passaggi di parametri tramite "value":

```
1 void swap (value int a, value int b) {
2     int temp = a;
3     a = b;
4     b = temp;
5 }
6 ...
7 int c = 3;
8 int d = 4;
```

```
9 swap(c, d);
```

Durante l'esecuzione di questo codice abbiamo la memoria allocata per c e d con i valori 3 e 4 rispettivamente.

$$c \rightarrow 3, d \rightarrow 4$$

Alla chiamata di $swap$ viene allocata un RdA per a , b e per $temp$. I valori di a e b vengono inizializzati con i valori di c e d rispettivamente, quindi:

$$a \rightarrow 3 \quad b \rightarrow 4 \quad temp \rightarrow \text{non inizializzato}$$

Esecuzione di swap:

```
1 tmp -> 3
2 a -> 4
3 b -> 3
```

Alla terminazione di $swap$ viene distrutto l'RdA e le modifiche a a e b vengono perse e quindi la memoria di c e d rimane invariata.

5.13.3 Passaggio per riferimento

Definizione 5.22: Passaggio per riferimento

Viene passato un cammino di accesso di parametro formale.

- Come vantaggio ha un uso efficiente della memoria.
- Come svantaggio crea degli alias.

Esempio 5.31

Consideriamo il seguente codice; facciamo finta che si possa specificare il tipo di passaggi di parametri tramite "reference":

```
1 void foo (reference int x) { x = x + 1; }
2 ...
3 int y = 1;
4 foo(y); // Notare che ora abbiamo un l-value quindi non possiamo applicare l'
          'operatore + a y direttamente
```

- Alla chiamata si crea RdA con cella per x
- Alla chiamata x diventa un riferimento a y e quindi x e y sono alias
- Alla terminazione viene distrutto RdA contenente il riferimento x

- Tutte le modifiche fatte a x durante l'esecuzione di foo rimangono in memoria e quindi y viene modificato a 2.

Esempio 5.32

Consideriamo il seguente codice:

```
1 void swap (reference int a, reference int b) {
2     int temp = a;
3     a = b;
4     b = temp;
5 }
6 ...
7 int c = 3;
8 int d = 4;
9 swap(c, d);
```

Esecuzione:

- Eseguiamo il main e abbiamo:

$$c \rightarrow 3, d \rightarrow 4$$

- Alla chiamata di *swap* viene allocata un RdA per a , b e per $temp$.

$$tmp \rightarrow [] \quad a \rightarrow c \quad b \rightarrow d$$

- Esecuzione di *swap*:

```
1 tmp -> 3
2 a -> 4 <- c
3 d -> 3 <- b
4
```

- Alla terminazione di *swap* viene distrutto l'RdA e le modifiche a a e b rimangono in memoria e quindi c e d vengono modificati a 4 e 3 rispettivamente.

Esempio 5.33

Consideriamo il seguente codice javascript:

```
1 > a = { val: 2 }
2 { val: 2 }
3 > b = { val : 3 }
4 { val: 3 }
5 > function double (x,y) {
6 | x.val = 2 * x.val;
```

```

7 | y.val = 2 * y.val;
8 | console.log(x.val + ' ' + y.val)
9 | }
10 undefined
11 > console.log(double(a,b))
12 4 6
13 undefined
14 undefined
15 > console.log(a.val + ' ' + b.val)
16 4 6
17 undefined
18 >

```

Come possiamo notare stampare le variabili all'interno della funzione e quelle globali, il valore non cambia.

5.14 Funzioni di ordine superiore

Le funzioni di ordine superiori sono funzioni passate come parametro ad altre funzioni. Esiste anche la controparte, restituire una funzione come risultato ma non ne parleremo poiché abbastanza complesso. Il problema a cui ci riferiamo è la risoluzione di riferimenti non locali in procedure / funzioni passate come parametro ad altre procedure / funzioni.

5.14.1 Regole di Binding

In particolare nel caso **dello scoping dinamico** sono necessarie anche delle regole aggiuntive dette di binding.

- **Deep Binding:** Utilizzo l'ambiente della creazione del legame tra attuale e formale
- **Shallow Binding:** Utilizzo l'ambiente della chiamata della funzione mediante il parametro formale

Esempio 5.34

Consideriamo il seguente codice:

```

1 int x = 4; int z = 0; // ambiente valido al momento della definizione della
    procedura con ambiente non locale
2 int f(int y) { // (1)
3     return x * y;
4 }
5 void g(int h(int n)) {
6     int x = 7; // ambiente valido quando la procedura viene effettivamente
    eseguita mediante parametro formale
7     z = h(3) + x; // (3)
8 }

```

```

9 ...
10 {
11     int x = 5; // ambiente valido al momento del passaggio di f
12     g(f); // (2) f non locale
13 }

```

Notiamo come abbiamo una procedura f con ambiente non locale. Abbiamo bisogno almeno di regole di scoping e potenzialmente di binding. Abbiamo tre ambienti possibili dove x è definita e quindi dobbiamo risolverla.

Essendo che viene passata f a g con ambiente non locale allora servono le regole di binding. Entrambi sono ambienti determinabili a tempo di esecuzione quindi sono rilevanti in caso di scoping dinamico.

- Se lo scoping è statico si utilizza l'ambiente (1) per tutto, per tutte le procedure anche quelle passate come parametro. Ambiente della definizione della procedura.
- Se è dinamico, per le procedure non passate come parametro ma con ambiente non locale
 - (2) Se l'ambiente della creazione del legame tra attuale e formale, per le procedure passate come parametro (**Deep Binding**)
 - (3) Se l'ambiente della chiamata della funzione mediante il parametro formale, per le procedure passate come parametro (**Shallow Binding**)

Torniamo al nostro esempio:

- Ambiente scoping statico: $x = 4$ L'esecuzione di questo ambiente è il seguente:
 - Si esegue la chiamata $g(f)$
 - Si esegue la chiamata $h(3)$ che è in realtà $f(3)$. x è locale e quindi $x = 7$
 - Si esegue $x * 3$, non locale e quindi scoping statico, prendo $x = 4$ e quindi $4 * 3 = 12$
- Ambiente scoping dinamico con deep binding: $x = 5$ L'esecuzione di questo ambiente è il seguente:
 - Eseguo la chiamata di $g(f)$
 - Eseguo la chiamata di $z = h(3) + x$ dove x è locale e quindi $x = 7$
 - Eseguo h e quindi $x * 3$, con deep binding prendo $x = 5$
 - Tornando indietro per le chiamate abbiamo quindi $15 + 7 = 22$
- Ambiente scoping dinamico con shallow binding: $x = 7$

- Chiamata di $g(f)$
- Chiamata di $z = h(3) + x$ dove x è locale e quindi $x = 7$
- Eseguo h e quindi $x * 3$, con shallow binding prendo $x = 7$
- Tornando indietro per le chiamate abbiamo quindi $21 + 7 = 28$

5.15 Ricorsione

La ricorsione è un metodo alternativo alla costruzione di procedure iterative.

Definizione 5.23

Una procedura è **ricorsiva** se all'interno della procedura esiste una chiamata alla procedura stessa.

Esempio 5.35

Consideriamo il seguente codice in IMP:

```

1 int fact (int n) {
2   if (n <= 0) {
3     return 1;
4   } else {
5     return n * fact(n - 1);
6   }
7 }
```

Esempio 5.36

Consideriamo il seguente codice in IMP:

```

1 int fun1(int n) {
2   if (n == 1) then return fun1(n);
3 }
4
5 int fun2(int n) {
6   if (n == 0) then return 1;
7   else return fun2(n + 1);
8 }
```

Entrambe le funzioni sono ricorsive, tuttavia non si possono considerare come un procedimento induttivo poiché queste funzioni non terminano mai.

- Nella prima funzione, quando n è uguale a 1, viene chiamata la funzione stessa con lo stesso valore di n , quindi si entra in un ciclo infinito.

- Nella seconda funzione, quando n è uguale a 0, viene restituito 1, ma se n è diverso da 0, viene chiamata la funzione stessa con un valore di n incrementato di 1, quindi si entra in un ciclo infinito che incrementa continuamente n .

L'induzione non è un'induzione se non ha una **terminazione** e quindi torno al caso base.

Ricorsione vs Iterazione

- L'iterazione è sempre possibile in qualunque PL turing completo
- Per la ricorsione è possibile se le procedure possono essere definite in termini di sé stesse.
 - Allocazione dinamica
 - Ricorsione in coda

Ogni programma iterativo può essere riscritto in modo ricorsivo e viceversa ogni programma iterativo può essere trasformato in forma ricorsiva.

Esempio 5.37

Consideriamo il seguente codice in IMP:

```
1 int fact (int n) {  
2   if (n <= 0) {  
3     return 1;  
4   } else {  
5     return n * fact(n - 1);  
6   }  
7 }
```

Se noi chiamiamo `fact(3)`, abbiamo la seguente sequenza di chiamate:

- `fact(3)` chiama `fact(2)`
- `fact(2)` chiama `fact(1)`
- `fact(1)` chiama `fact(0)`
- `fact(0)` restituisce 1 (caso base)
- `fact(1)` restituisce $1 * 1 = 1$
- `fact(2)` restituisce $2 * 1 = 2$
- `fact(3)` restituisce $3 * 2 = 6$
- Quindi il risultato finale è 6

5.15.1 Ricorsione in coda

Una chiamata ricorsiva è in coda se è l'ultima operazione eseguita dalla procedura prima di restituire un valore al chiamante.

- Chiamata in coda:
 - f chiama g
 - la chiamata di g è detta in coda se f restituisce il valore restituito da g senza ulteriori computazioni.
- f è ricorsiva in coda se chiama la chiamata ricorsiva a sè stessa è in coda.

Esempio 5.38

Guardiamo l'esempio sul fattoriale:

```
1 int fact(int n) {  
2     fact-rc(n, *); // <-- valore iniziale per res,  
3     // indicativamente e' il risultato  
4     // del caso base  
5 }  
6  
7 int fact-rc(int n, int res) {  
8     if (n == 1) return res;  
9     else fact-rc(n - 1, n * res);  
10 }
```

La sequenza delle chiamate è la seguente:

- $\text{fact}(3) \rightarrow \text{fact-rc}(3, 1)$
- $\text{fact-rc}(3, 1) \rightarrow \text{fact-rc}(2, 3)$
- $\text{fact-rc}(2, 3) \rightarrow \text{fact-rc}(1, 6)$
- $\text{fact-rc}(1, 6)$ restituisce 6 (caso base)
- Quindi il risultato finale è 6

Esempio 5.39 (Fibonacci)

Proviamo a scrivere la funzione di Fibonacci in modo ricorsivo e ricorsivo in coda:

```
1 int fib(int n) {  
2     if (n <= 0) return 0;  
3     else if (n == 1) return 1;  
4     else return fib(n - 1) + fib(n - 2);  
5 }  
6
```

La complessità esponenziale sia in tempo che in spazio a causa della somma di due chiamate ricorsive che a loro volta fanno altre chiamate ricorsive. La ricorsione in coda migliora questi costi per evitare la crescita esponenziale della pila di chiamate.

```
1 int fib(int n) {  
2     return fib-rc(n, 0, 1); // inizializziamo i valori per i primi due numeri  
   di Fibonacci (caso base)  
3 }  
4  
5 int fib-rc(int n, int a, int b) {  
6     if (n == 0) return a;  
7     else if (n == 1) return b;  
8     else return fib-rc(n - 1, b, a + b);  
9 }
```